

ADANI PUBLIC SCHOOL
MUNDRA · GUJARAT

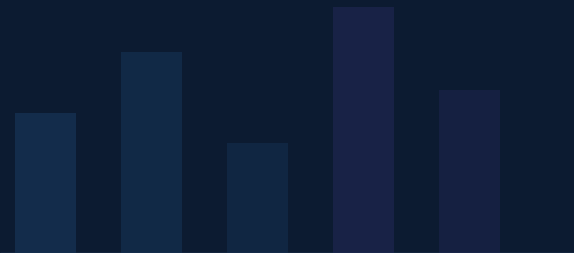
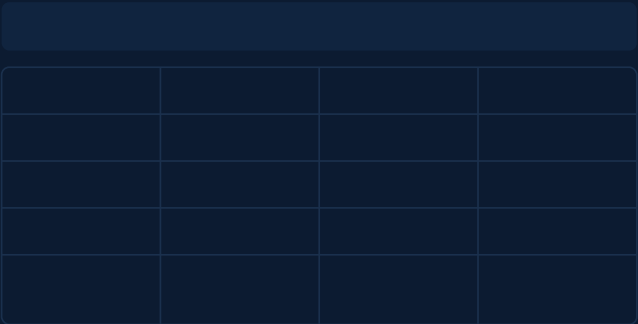
INFORMATICS PRACTICES · PROJECT REPORT

STUDENT MANAGEMENT SYSTEM

Student Records, Academic Analytics & Data Visualization

CLASS	XII	ACADEMIC SESSION	2026-27
SUBJECT	Informatics Practices	PROJECT TYPE	Python Desktop Application
DATA STORAGE	CSV File	CATEGORY	Student Record Management & Analytics

- Python
- CustomTkinter
- Tkinter
- Pandas
- Matplotlib
- CSV



Project Details

PROJECT TITLE	Student Management System
SUBTITLE	Student Records, Academic Analytics & Data Visualization
SUBJECT	Informatics Practices (CBSE)
CLASS	XII
ACADEMIC SESSION	2026-27
SCHOOL	Adani Public School, Mundra
PROJECT TYPE	Python Desktop Application
DATA STORAGE	CSV file (comma-separated values)
SOURCE FILE	Project Program.py
SAMPLE DATASET	Student.csv

Technology strip

Python

CustomTkinter

Tkinter

Pandas

Matplotlib

CSV File Handling

Student Name

Roll Number / Section

Certificate

ADANI PUBLIC SCHOOL, MUNDRA

CERTIFICATE

Informatics Practices (065) · Class XII · Academic Session 2026-27

This is to certify that the project titled

"Student Management System"

has been completed and submitted as the Informatics Practices project work of Class XII during the academic session 2026-27. The project has been developed as a Python desktop application for maintaining student records and generating academic analytics, and the accompanying report documents the same implementation.

Student Name

Class / Section

Roll Number

Teacher / Guide

Date

Teacher / Guide Signature

External / Examiner Signature

Declaration

I hereby declare that the project work titled **"Student Management System"** submitted for the Class XII Informatics Practices coursework of the academic session 2026-27 is my own work, developed as an academic project.

The application has been written in Python and uses Tkinter / CustomTkinter for the graphical interface, Pandas for tabular data processing, Matplotlib for charts, and CSV files for data storage. The source code, the sample dataset and the documentation presented in this report represent the work submitted for evaluation. Any external reference material consulted during development has been listed in the bibliography section of this report.

The project has been prepared for academic assessment and demonstration purposes only.

Student Name

Class / Section

Roll Number

Date

Place

Student Signature

Acknowledgement

I would like to express my sincere gratitude to my Informatics Practices teacher for the guidance, suggestions and continuous support provided throughout the development of this project. The explanation of Python programming concepts, database handling, data analysis with Pandas and data visualisation techniques helped me convert an idea into a working desktop application.

I am thankful to **Adani Public School, Mundra** for providing the academic environment, laboratory facilities and encouragement required to complete this project work, and to the subject teachers who reviewed the progress of the application at different stages.

I also acknowledge the guidance received while learning the standard documentation of Python, Pandas, Matplotlib and CustomTkinter, which made it possible to design the interface, process the student data and generate charts without external help.

The project gave me an opportunity to learn how a real application is planned, coded, tested, and documented. The process of identifying errors, correcting validation logic and improving the interface design has been the most valuable learning experience of this work.

Student Name

Class / Section

Date

Signature

Abstract

A concise technical overview of the submitted application

The **Student Management System** is a desktop application developed in Python that maintains student academic records and produces analytical reports from them. The project was developed as the Class XII Informatics Practices project work for the academic session 2026-27.

Student information such as roll number, name, class, section, gender and age, together with the marks obtained in English, Physics, Chemistry and Mathematics, is stored in a **CSV (comma-separated values) file**. This file acts as the single persistent storage unit of the application. When the program starts, it checks whether the data file exists, and creates an empty file with the predefined column structure if the file is not present.

The graphical interface of the application is built using **CustomTkinter**, which extends the standard **Tkinter** toolkit with a modern, dark appearance. The interface contains an information header, four dashboard cards, a student details form, a record table, a search area, ten analytics buttons and three chart buttons. All record operations are performed from this single window, so the user does not need to open the CSV file manually at any stage.

The application supports complete **CRUD (Create, Read, Update, Delete)** functionality. A new student record can be added after validating every input field; an existing record can be updated by entering its roll number; a record can be deleted after a confirmation dialog; and all records are displayed in a scrollable table. In addition, the program can **import an external CSV file** containing student data. During import, the program checks for missing columns, converts numeric columns, rejects records containing invalid or missing values, and refuses to import roll numbers that already exist in the database.

Pandas is used for all tabular processing. The CSV file is read into a DataFrame, new records are appended using concatenation, updates are performed with conditional selection, and deletions are carried out by filtering the DataFrame. The same DataFrame is used for the analytical calculations, which include class-wise average percentage, topper and lowest scorer, the top five and bottom five students, subject-wise averages, pass count, fail count, pass percentage and grade distribution.

Matplotlib is used to generate three charts from the loaded data: a bar chart comparing the top five students by percentage, a bar chart of subject-wise average marks, and a pie chart showing the distribution of grades. The dashboard area of the window displays four key indicators that are recalculated every time the data changes: total number of students, average percentage, the name of the topper and the pass percentage.

Validation is an important part of the implementation. The program verifies that no field is left blank, that the roll number and age are integers, that the age lies between 1 and 100, that marks are numeric values between 0 and 100, and that a roll number is not duplicated. Total marks, percentage and grade of every student are calculated automatically by the program, so the stored data stays internally consistent.

The application is an offline, single-user desktop program. It does not use a database server, cloud storage, network communication or user authentication, and all analytical results are computed from the records currently present in the CSV file. The dataset supplied with this report contains twenty dummy student records that are used for demonstration and testing.

In one line: a Python desktop application that stores student records in a CSV file, manages them through a CustomTkinter interface using Pandas, and presents academic analytics as computed results and Matplotlib charts.

Table of Contents

Chapter-wise contents with page references

FRONT MATTER

Project Title Page	2
Certificate	3
Declaration	4
Acknowledgement	5
Abstract	6
Table of Contents	7
1. Introduction	10
2. Objectives	12
3. Technology Stack	13
4. System Requirements	16
5. Project Architecture	18
6. Data Model and CSV Structure	20
7. Grade Calculation Logic	22
8. Application Features	24
8.1 Dashboard	24
8.2 Add Student	24
8.3 Update Student	25
8.4 Delete Student	25
8.5 Import CSV	25
8.6 Search	25
8.7 View All	26
8.8 Automatic Calculations	26
8.9 Analytics	26
8.10 Graphical Visualisation	26

9. User Interface Design	28
10. Program Logic	31
11. Input Validation	34
12. CRUD Operations	37
13. Data Import Workflow	39
14. Search Workflow	42
15. Analytics Module	44
16. Data Visualisation	47
17. Sample Dataset	50
18. Sample Analytical Results	53
19. Application Output	55
19.1 Main Dashboard Output	55
19.2 Populated Student Table	55
19.3 Search Result View	56
19.4 Analytics Result Dialogs	56
19.5 Chart Windows	57
20. Testing	59
21. Error Handling	61
22. Algorithm and Logical Flow	63
23. Pseudocode	66
APPENDICES	
Appendix A — Complete Source Code	70
A.1 Setup and Configuration	70
A.2 Helper Functions	71
A.3 CRUD Operations	73
A.4 Analytics and Graph Functions	75
A.5 GUI Layout Design	76
Appendix B — Sample Dataset	80

CLOSING SECTIONS

26. Project Limitations	83
27. Future Enhancements	84
28. Conclusion	85
29. Learning Outcomes	86
30. Bibliography and References	87
31. Final Project Summary	88

Note — Page numbers refer to the printed page numbers shown at the bottom of every page of this report. Appendices A and B carry the complete source code and the supplied sample dataset respectively.

CHAPTER 01

Introduction

Background, problem statement, proposed solution, scope and intended users

1.1 Background

Every school maintains a large amount of information about its students — roll numbers, names, classes, sections, age and the marks obtained in different subjects. When this information is kept in registers or in loose sheets of paper, using it for any purpose other than reading a single entry becomes difficult. Marks have to be totalled by hand, percentages have to be calculated again whenever a correction is made, and preparing a summary such as “which section has the highest average” requires a fresh round of manual work.

Computerised record management replaces this manual process with a structured digital file that stores the same information in a fixed, machine-readable format. Once the data is in a structured form, the computer can perform the repetitive arithmetic, keep the derived values (total, percentage, grade) consistent with the subject marks, and produce summaries and charts within seconds. Informatics Practices introduces precisely these ideas — file handling, tabular data processing with Pandas, and graphical user interface design — which is why a student record system is a suitable problem for a Class XII project.

1.2 Problem Statement

The manual method of handling student records suffers from a set of practical limitations:

- **Repetitive calculations** — total marks, percentage and grade have to be calculated by hand for every student, and recalculated whenever a single subject mark changes.
- **Difficulty in searching records** — locating a particular student in a register requires scanning the pages one by one; searching by name is even slower because the records are usually arranged by roll number.
- **Manual updating** — a correction in one student’s marks must be carried forward to the total, the percentage and the grade, and errors of omission are common.
- **Difficulty in producing statistics** — questions such as the class average, the number of students who passed, or the grade distribution cannot be answered quickly from a register.
- **Lack of visual analysis** — a register cannot show at a glance which students are at the top, how the averages of the four subjects compare, or how the grades are spread across the class.
- **Risk of inconsistent data** — the same value may be written differently in two places, and nothing prevents two students from being entered with the same roll number.

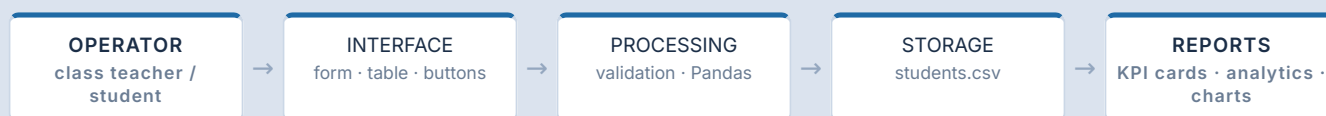
1.3 Proposed Solution

The **Student Management System** is a single-window desktop application that addresses these problems by combining four ideas:

1. **Centralised storage** — all student records are kept in one CSV file (`students.csv`) with a fixed set of thirteen columns. The program creates this file automatically with the correct column headings if it does not already exist.
2. **A graphical interface** — a form, buttons, a search box and a scrollable table built with CustomTkinter and Tkinter allow the records to be entered, corrected, searched and displayed without ever opening the CSV file in a spreadsheet.
3. **Automatic derived values** — as soon as the marks of a student are entered, the program computes the total marks, the percentage and the grade, so the file never stores an inconsistent record.

4. **Built-in analysis and visualisation** — ten analytics operations and three Matplotlib charts turn the stored records into summaries such as the class average, the topper, pass percentage and grade distribution.

The result is a self-contained application in which data entry, storage, calculation, searching, reporting and visualisation all happen from a single interface.



1.4 Scope of the Project

The submitted implementation handles the following work:

- storing student records in a CSV file with a predefined thirteen-column structure;
- adding, updating and deleting records through the interface, with validation before the file is written;
- displaying all records in a scrollable table and filling the input form directly from the selected row;
- searching records by name or roll number;
- importing an external CSV file of student records, after checking its columns and values;
- automatically calculating total marks, percentage and grade for each student;
- displaying four dashboard indicators — total students, average percentage, topper name and pass percentage;
- computing ten analytical summaries from the stored data;
- drawing three Matplotlib charts — top five students, subject averages and grade distribution.

The scope does not include attendance, fees, timetable, examination scheduling, user accounts, or any form of networked or multi-user access. These are outside the purpose of the present implementation.

1.5 Intended Users

The application is designed for a single operator working at one computer, such as a class teacher or a member of the school's academic staff who maintains marks data. During the demonstration and evaluation of this project, the same role is performed by the student presenting the work. The program is an academic project: it models the working of a school record system and is not an official software deployment of the school.

Figure 1.1 — Context of the application — the operator enters records through the interface; the program validates them, stores them in the CSV file and produces the reports from the same data.

CHAPTER 02

Objectives

What the finished application is expected to accomplish

The objectives of the project define what the finished application is expected to do. Each objective listed below is implemented in the submitted source code and can be demonstrated from the running program.

1. Record maintenance

Store every student record permanently in a structured CSV file with a fixed set of thirteen columns so that data survives between program runs.

2. Adding records

Provide an input form that accepts a new student's details and appends the record to the data file only after all fields have been validated.

3. Updating records

Allow an existing student's record to be located by roll number and modified without disturbing any other record in the file.

4. Deleting records

Remove a record that is no longer required, only after the operator confirms the deletion, so that accidental loss of data is avoided.

5. CSV import

Load student data from an external CSV file, checking that all required columns are present, that numeric fields are valid and that no roll number is duplicated.

6. Searching

Retrieve matching records instantly by entering either a student's name or a roll number in the search box.

7. Total marks

Calculate the sum of the four subject marks automatically for every student instead of relying on manual addition.

8. Percentage

Derive the percentage from the total marks of the four subjects, so the value always matches the subject marks that are stored.

9. Grade

Assign a grade using fixed percentage thresholds so that the same percentage always produces the same grade.

10. Dashboard indicators

Keep four indicator cards on screen — total students, average percentage, topper name and pass percentage — always in step with the current data.

11. Academic analytics

Produce ten analytical summaries including class average, topper, lowest scorer, top five, bottom five, subject averages, pass and fail counts, pass percentage and grade distribution.

12. Data visualisation

Present key results graphically as bar charts and a pie chart generated with Matplotlib from the records currently stored.

Table 2.1 — The objectives of the project, grouped as record management, computation and reporting.

KEY POINT — OBJECTIVE VS. FEATURE

An objective is a statement of what the project is expected to achieve; a feature is the part of the program that achieves it. Every objective listed on this page maps to a named function in the source code, which is documented in Chapter 10.

KEY POINT — SCOPE CONTROL

The objectives deliberately exclude attendance, fees, user accounts and network access, because the submitted implementation does not provide them. Keeping the objective list honest is part of accurate documentation.

CHAPTER 03

Technology Stack

The libraries and storage format actually used by the project

Five Python libraries and one storage format are used in the project. All of them appear in the import section of the source file, and no other library is used anywhere in the program.

```
1 import os
2 import tkinter as tk
3 from tkinter import ttk, messagebox
4 import customtkinter as ctk
5 import pandas as pd
6 import matplotlib.pyplot as plt
```

Source: `Project Program.py`, lines 1-6 — the complete import section of the project.

Figure 3.1 — Technology layers of the application: the interface toolkit on top, data processing and storage in the middle, analysis and plotting at the base.

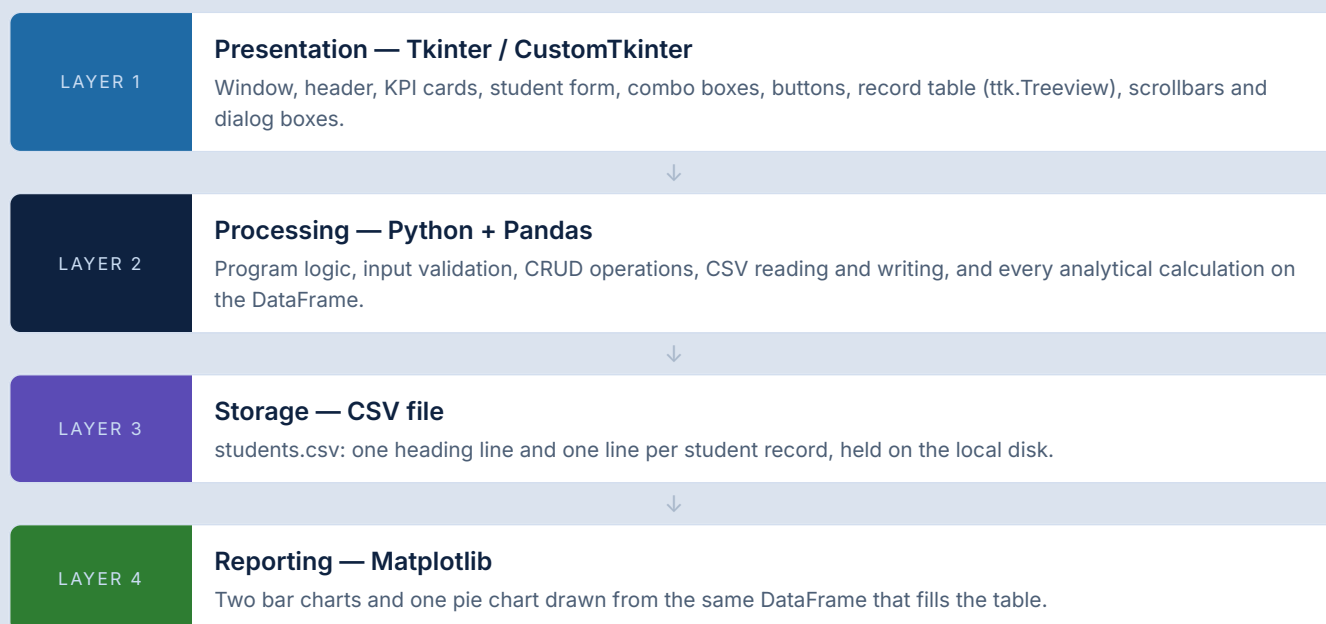


Figure 3.2 — Technology stack of the project, from the interface at the top to the reporting layer.

3.1 Python

Python is the programming language in which the entire application is written. It provides the program structure, the functions that carry out each operation, the exception handling used for validation, and the file handling through which the CSV data file is managed.

Aspect	Role in this project
Purpose	General-purpose programming language used to write the complete application, from the storage functions to the interface.
Role in the application	Defines every function of the program — <code>calculate_grade()</code> , <code>add_student()</code> , <code>import_csv()</code> , <code>show_analytics()</code> and the rest — together with the <code>try / except</code> blocks that report invalid input.
Contribution to the project	Allows data validation, arithmetic, file handling and program flow to be written in short, readable functions, so a change in one operation does not disturb the others.

3.2 Tkinter

Tkinter is the standard graphical user interface toolkit that ships with Python. In this project it supplies the record table and the message and file dialogs, while the overall window styling is handled by CustomTkinter.

Aspect	Role in this project
Purpose	Provides the fundamental windowing toolkit of the standard library.
Role in the application	<code>ttk.Treeview</code> displays the thirteen columns of student data; <code>ttk.Scrollbar</code> adds vertical and horizontal scrolling; <code>ttk.Style</code> applies the dark colours of the table; <code>messagebox</code> shows information, warning and error dialogs and asks for delete confirmation; <code>tk.filedialog</code> opens the file chooser used by the CSV import; <code>tk.StringVar</code> links the ten form fields to Python variables.
Contribution to the project	Makes the application interactive: a table that scrolls, dialogs that inform the user, and variables that keep the interface and the data in step.

3.3 CustomTkinter

CustomTkinter is a modern wrapper around Tkinter that provides rounded, themable widgets. The application uses it for the window itself, the frames, the labels, the input fields, the combo boxes, the KPI cards and every button.

Aspect	Role in this project
Purpose	Supplies modern interface widgets with a consistent appearance and a built-in dark theme.
Role in the application	<code>ctk.set_appearance_mode("Dark")</code> switches the whole window to the dark appearance and <code>ctk.set_default_color_theme("blue")</code> selects the blue accent theme. <code>CTkFrame</code> , <code>CTkLabel</code> , <code>CTkEntry</code> , <code>CTkComboBox</code> and <code>CTkButton</code> build the header, the four KPI cards, the student form, the search bar, the analytics grid and the chart buttons.
Contribution to the project	Gives the application a single, consistent visual language: one function call changes the appearance mode, and every widget follows it.

3.4 Pandas

Pandas is the data analysis library that performs all tabular work in the project. Every read, write, search, update, delete and analytical calculation passes through a Pandas DataFrame.

Aspect	Role in this project
Purpose	Loads CSV files into a DataFrame and provides ready-made operations for selecting, filtering, combining, sorting and summarising tabular data.
Role in the application	<code>pd.read_csv()</code> loads the data file; <code>pd.concat()</code> appends a new record or an imported file; <code>df.loc[]</code> updates the values of one student; boolean filtering removes a deleted record; <code>df.to_csv()</code> saves the modified DataFrame; <code>mean()</code> , <code>groupby()</code> , <code>nlargest()</code> , <code>smallest()</code> and <code>value_counts()</code> produce the analytics; <code>pd.to_numeric()</code> converts imported columns; <code>isnull()</code> detects missing values.
Contribution to the project	Reduces the data-handling part of the program to a few lines per operation, and guarantees that the same DataFrame logic drives both the table and the analytics.

3.5 Matplotlib

Matplotlib is the plotting library used for the three charts of the project. The charts open in their own window and are drawn from the same CSV data that the table displays.

Aspect	Role in this project
Purpose	Creates publication-style static charts from Python data structures.
Role in the application	<code>plt.figure(figsize=(8, 5))</code> creates the figure; <code>plt.bar()</code> draws the top-five chart and the subject-average chart; <code>plt.pie(..., autopct="%1.1f%%", startangle=140)</code> draws the grade distribution pie; <code>plt.title()</code> , <code>plt.xlabel()</code> and <code>plt.ylabel()</code> label the charts, and <code>plt.show()</code> displays the final figure.
Contribution to the project	Converts numerical summaries into visual form, making comparisons such as performance across subjects immediately understandable.

3.6 CSV File Storage

The project stores its data in a CSV file named `students.csv`, created in the same folder as the program.

Aspect	Role in this project
Purpose	A plain-text format in which each line is one record and the values are separated by commas.
Role in the application	Acts as the only storage layer. The first line of the file holds the thirteen column names; every following line holds one student's record, including the calculated total, percentage and grade.
Contribution to the project	The file is human-readable, opens directly in a spreadsheet application for inspection, and can be copied or replaced easily — which also makes the CSV import feature possible.

```

14
15 FILE_NAME = os.path.join(os.path.dirname(__file__), "students.csv")

```

Source: `Project Program.py`, lines 14–15 — the two calls that define the appearance of every widget in the application.

Listing 3.1 — The two configured properties that control the appearance of the entire interface.

KEY POINT — WHY SIX TECHNOLOGIES AND NOT ONE?

Each library answers a different question — Tkinter and CustomTkinter draw the interface, Pandas handles the table, Matplotlib draws charts and CSV stores the data. Python is the language that ties them together. This division is why the program can stay under 550 lines while offering a full CRUD application with analytics.

CHAPTER 04

System Requirements

Hardware and software required to run the application

The requirement lists below describe a typical computer on which the application runs comfortably. The project itself does not contain a formal configuration file, so these figures are recommended working values rather than mandatory specifications stated in the source code.

4.1 Hardware Requirements

Component	Requirement	Remark
Processor	Any modern 64-bit dual-core processor (Intel / AMD) or equivalent	Recommended — the program does simple arithmetic and table operations.
Memory (RAM)	4 GB or more	Recommended — Pandas and Matplotlib load a small dataset into memory.
Free disk space	About 500 MB for Python and the packages, plus a few kilobytes for the CSV file	Typical requirement of the Python installation.
Display	A desktop resolution of at least 1366 × 768	The application window is created with the size 1400 × 800, so a wider screen displays it without scaling.
Input devices	Keyboard and mouse	Required to fill the form and operate the buttons.

Table 4.1 — Recommended hardware configuration for running the application.

4.2 Software Requirements

Software	Requirement / installation	Purpose in the project
Operating system	Windows 10 / 11 or a modern Linux desktop distribution	The program uses only cross-platform libraries; no operating-system specific feature is used.
Python interpreter	Python 3.9 or later	The source uses the f-string style formatting and modern function syntax.
customtkinter	Install with <code>pip install customtkinter</code>	Provides every modern widget used by the interface.
pandas	Install with <code>pip install pandas</code>	Required for all table operations and analytics.
matplotlib	Install with <code>pip install matplotlib</code>	Required by the three chart functions.
tkinter	Normally included with the standard Python installer on Windows and macOS; on some Linux distributions it is supplied by a separate package such as <code>python3-tk</code>	Supplies the record table, the scrollbars and the dialog boxes.

Table 4.2 — Software stack required by the program, as identified from the import statements.

KEY POINT — WHY ARE THE REQUIREMENTS 'RECOMMENDED' VALUES?

The source code does not declare any minimum configuration. The values given here are the practical requirements of Python, Pandas, Matplotlib and a graphical window, and are labelled accordingly so that the report does not claim a specification that the project never states.

4.3 Setting up the environment

The application can be prepared for a demonstration in three steps. The commands below are the standard pip commands for the libraries that appear in the import section of the program.

```
1 python -m pip install customtkinter pandas matplotlib
2 cd "folder containing Project Program.py"
3 python "Project Program.py"
```

The three commands used to install the dependencies and start the application on a computer that already has Python and Tkinter available.

KEY POINT — WHICH PACKAGES ARE ESSENTIAL?

customtkinter, pandas and matplotlib must be installed separately. tkinter and os are part of the standard Python installation on Windows and macOS; on some Linux distributions Tkinter is provided by a separate system package.

CHAPTER 05

Project Architecture

How the layers of the application are connected

The application follows a layered design. Every request from the user passes through the interface to the operations layer, then to Pandas, then to the CSV file — and the result travels back the same way. The four dashboard indicators sit beside this path and are refreshed from the DataFrame whenever the table is redrawn.

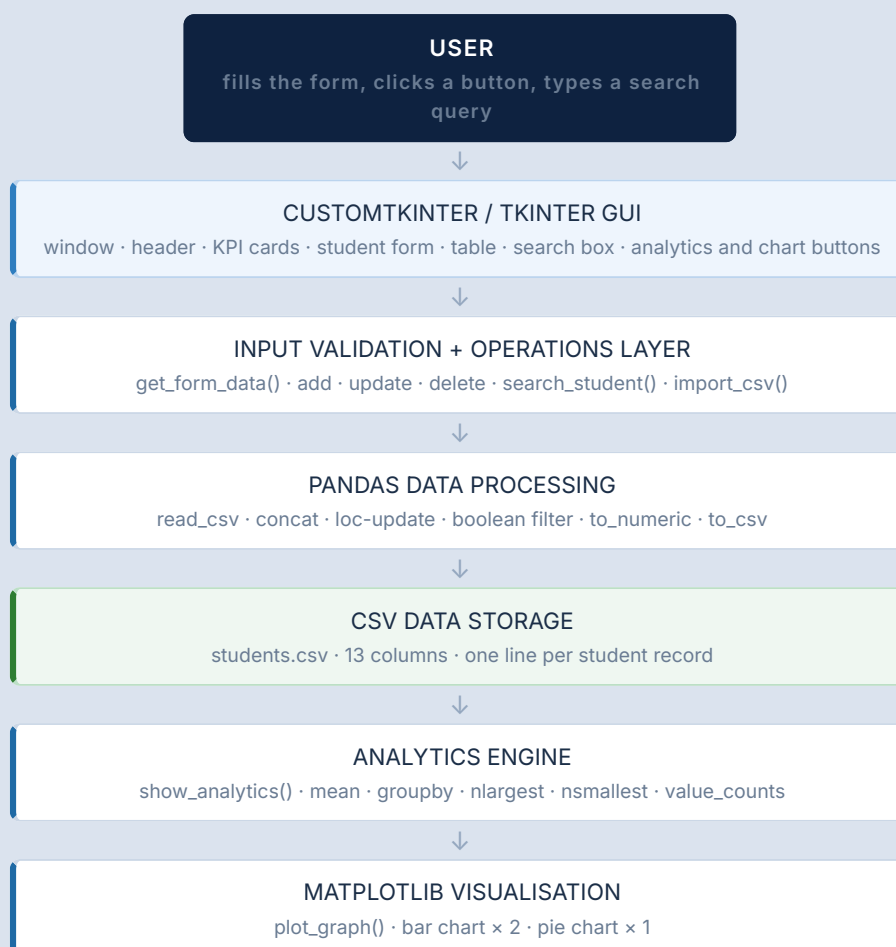


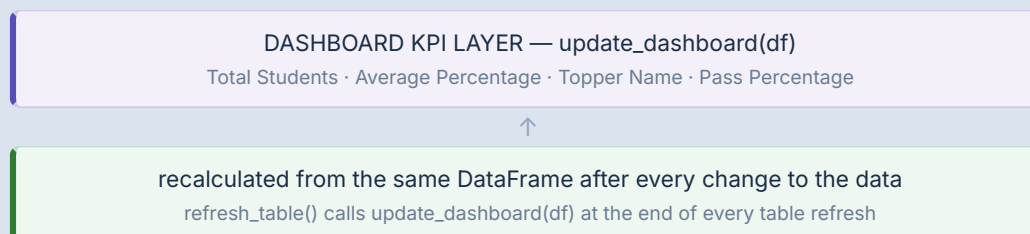
Figure 5.1 — System architecture of the Student Management System as implemented in the source code.

5.1 Layers and their responsibilities

Layer	Module / function	Responsibility
Presentation	CustomTkinter window, CTK widgets, ttk.Treeview	Displays data and collects user input; contains no data logic of its own.
Operations	add_student, update_student, delete_student, search_student, import_csv, get_form_data	Validates input and decides what must happen to the data.
Data processing	Pandas DataFrame operations	Reads, combines, filters and summarises records.
Storage	students.csv	Persists every record, including the calculated total, percentage and grade.

Layer	Module / function	Responsibility
Analytics	show_analytics(), plot_graph()	Computes the ten summaries and draws the three charts.
Dashboard / KPI	update_dashboard(), create_kpi_card()	Recalculates and displays the four indicator values after each refresh.

Table 5.1 — Layer-wise responsibilities and the functions that implement them.



5.2 Data flow of a typical operation

The diagram below follows a single change of data — for example adding a student — through the layers of the architecture. Every operation of the application uses this same path.

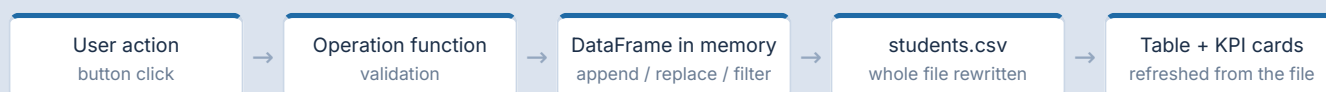


Figure 5.2 — Data flow of a typical operation through the layers of the application.

KEY POINT — WHAT IS NOT PART OF THE ARCHITECTURE

The design contains no server, cloud database, API, login screen, internet connection or SQL database, because none of these is present in the submitted source code. The only persistent component is the CSV file on the local disk.

CHAPTER 06

Data Model / CSV Structure

How student records are organised inside the data file

The project stores all records in a single CSV file. The file is created automatically the first time the program runs, with the thirteen column headings listed in the `COLUMNS` constant of the source code.

6.1 Storage file

File name	<code>students.csv</code>
Location	In the same folder as the program, derived with <code>os.path.join(os.path.dirname(__file__), "students.csv")</code>
Created	Automatically at start-up if the file does not already exist, with the column headings but no data rows
Format	Plain CSV — first line contains the column names, each later line contains one student record
Sample dataset	<code>Student.csv</code> , the twenty-record dummy dataset supplied with this report; it is loaded through the Import CSV button

```

1 Roll Number,Name,Class,Section,Gender,Age,English,Physics,Chemistry,Mathematics,Total
  Marks,Percentage,Grade
2 101,Aarav Sharma,12,A,Male,17,85,92,88,90,355,88.75,A
3 102,Priya Patel,12,A,Female,17,78,85,82,79,324,81.00,A
4 : (one further line for every remaining student record)

```

Listing 6.1 — the first records of the supplied `Student.csv`, showing the heading line and one record per line.

Figure 6.1 — Physical organisation of the data file — one heading line followed by one line per student.

6.2 Column structure

Column	Type	Meaning / rule	Example
Roll Number	Integer	Unique identifier of a student; used to locate, update and delete records.	101
Name	Text	Full name of the student as entered in the form.	Aarav Sharma
Class	Text (11 / 12)	Class level of the student, selected from a combo box.	12
Section	Text (A / B / C / D)	Section of the class, selected from a combo box.	A
Gender	Text (Male / Female / Other)	Recorded gender of the student.	Male
Age	Integer	Age of the student in years; validated to lie between 1 and 100.	17
English	Number	Marks obtained in English, out of 100.	85
Physics	Number	Marks obtained in Physics, out of 100.	92
Chemistry	Number	Marks obtained in Chemistry, out of 100.	88

Column	Type	Meaning / rule	Example
Mathematics	Number	Marks obtained in Mathematics, out of 100.	90
Total Marks	Number	Calculated as English + Physics + Chemistry + Mathematics.	355
Percentage	Number	Calculated as Total Marks / 4.	88.75
Grade	Text (A+ to F)	Derived from the percentage using the fixed grade thresholds.	A

Table 6.1 — The thirteen columns of the data file and the meaning of each field.

```

17     "Roll Number",
18     "Name",
19     "Class",
20     "Section",
21     "Gender",
22     "Age",
23     "English",
24     "Physics",
25     "Chemistry",
26     "Mathematics",
27     "Total Marks",
28     "Percentage",
29     "Grade",
30 ]
31

```

Source: [Project Program.py](#), lines 17-31 — the column definition and the automatic creation of the data file.

KEY POINT — WHY CSV INSTEAD OF A DATABASE?

A CSV file needs no server, no installation and no connection string, and it can be opened in any spreadsheet program for inspection. For a single-user application that stores a few hundred records, it keeps the project simple and fully readable — which is also why the file can be replaced during the CSV import.

CHAPTER 07

Grade Calculation Logic

How total marks, percentage and grade are derived

Grades are never typed by the user. They are derived from the percentage by the function `calculate_grade()`, which is called while the record is being prepared, so the grade always matches the marks that are stored.

7.1 Percentage from four subjects

Each subject is marked out of 100 and four subjects are recorded: English, Physics, Chemistry and Mathematics. The total marks are the sum of the four subject marks, and the percentage is the total divided by four:

$$\text{Percentage} = (\text{English} + \text{Physics} + \text{Chemistry} + \text{Mathematics}) \div 4$$

$$\text{Total Marks} = \text{English} + \text{Physics} + \text{Chemistry} + \text{Mathematics} \cdot \text{Percentage} = \text{Total Marks} / 4.0$$

Step	Calculation	Worked example
Subject marks	English, Physics, Chemistry, Mathematics	85 + 92 + 88 + 90
Total marks	Sum of the four subject marks	355
Percentage	Total marks ÷ 4	355 / 4 = 88.75
Grade	<code>calculate_grade(88.75)</code>	A

Table 7.1 — Calculation sequence followed by the program for every student record.

7.2 Grade thresholds used by the program

Grade	Percentage rule	Range	Description	Records in sample data
A+	90 and above	90 - 100	Outstanding	4
A	80 to 89.99	80 - 89.99	Very good	8
B	70 to 79.99	70 - 79.99	Good	2
C	60 to 69.99	60 - 69.99	Satisfactory	2
D	40 to 59.99	40 - 59.99	Pass	3
F	Below 40	0 - 39.99	Fail	1

Figure 7.2 — Grade table exactly as implemented in `calculate_grade()`. The last column shows how many records of the supplied sample dataset fall in each grade.

```

45 def calculate_grade(percentage):
46     if percentage >= 90:
47         return "A+"
48     elif percentage >= 80:
49         return "A"
50     elif percentage >= 70:
51         return "B"
52     elif percentage >= 60:
53         return "C"
```

```
54     elif percentage >= 40:
55         return "D"
56     else:
57         return "F"
58
59
```

Source: `Project Program.py` — complete grading function. The thresholds 90, 80, 70, 60 and 40 are used in this order.

7.3 Validation of subject marks

Before the grade is calculated, each of the four subject marks is checked. A mark below 0 or above 100 is rejected with an error dialog, and the record is not saved. The age entered in the form is checked in the same way and must be greater than 0 and not more than 100. Because these checks run before the record is written, the CSV file can never contain a mark outside the 0-100 range.

```
135     if age <= 0 or age > 100:
136         messagebox.showerror("Error", "Please enter a valid age.")
137         return None
138
139     for mark in [eng, phy, chem, math]:
140         if mark < 0 or mark > 100:
141             messagebox.showerror("Error", "Marks must be between 0 and 100.")
142             return None
143
144     total = eng + phy + chem + math
145     percentage = total / 4.0
146     grade = calculate_grade(percentage)
```

Source: `Project Program.py` — age and marks validation inside `get_form_data()`.

KEY POINT — HOW IS THE GRADE CALCULATED?

The program calls `calculate_grade(percentage)` with the percentage of the student. The function tests the value against 90, 80, 70, 60 and 40 from the highest side downwards and returns the first grade that matches; if none matches, it returns "F". Because the comparison starts from the highest threshold, a percentage of 90 receives A+ and not A.

KEY POINT — WHY IS THE FAIL GRADE USED IN THE ANALYTICS?

The analytics treat every grade other than F as a pass: `df[df["Grade"] != "F"]`. The pass percentage and the pass count are therefore based directly on the grade stored in the file, and not on a separate rule.

CHAPTER 08

Application Features

Feature-by-feature documentation of the working application

The application offers ten feature areas, all reachable from the main window. Each feature below is described from the implemented behaviour in the source code.

8.1 Dashboard

Four indicator cards are displayed below the title bar. They are created by the helper function `create_kpi_card(parent, title)` and are refreshed by `update_dashboard(df)` every time the table is redrawn.

KPI card	Value shown and how it is calculated
Total Students	The number of records in the DataFrame — <code>len(df)</code> .
Average Percentage	Mean of the Percentage column — <code>df["Percentage"].mean()</code> , displayed with two decimal places.
Topper Name	Name of the student with the highest percentage — <code>df.loc[df["Percentage"].idxmax()]["Name"]</code> .
Pass Percentage	Students whose grade is not F, divided by the total number of students, expressed as a percentage and shown with two decimal places.

When the data file is empty, the four cards display 0, 0.0%, N/A and 0.0% respectively instead of failing.

```

91 def update_dashboard(df):
92     if df.empty:
93         kpi_total.configure(text="0")
94         kpi_avg.configure(text="0.0%")
95         kpi_topper.configure(text="N/A")
96         kpi_pass.configure(text="0.0%")
97         return
98
99     total_students = len(df)
100     avg_pct = df["Percentage"].mean()
101     topper_name = df.loc[df["Percentage"].idxmax()]["Name"]
102     pass_count = len(df[df["Grade"] != "F"])
103     pass_pct = (pass_count / total_students) * 100
104
105     kpi_total.configure(text=str(total_students))
106     kpi_avg.configure(text=f"{avg_pct:.2f}%")
107     kpi_topper.configure(text=str(topper_name))
108     kpi_pass.configure(text=f"{pass_pct:.2f}%")
109
110

```

Source: [Project Program.py](#) — the dashboard refresh function, including the empty-data case.

Table 8.1 — The four dashboard indicators and the Pandas expression behind each value.

8.2 Add Student

The form on the left side of the window contains ten input controls. When Add is clicked, the values are collected by `get_form_data()`, which validates every field, converts the roll number and age to integers and the marks to numbers, calculates the total, percentage and grade, and returns a dictionary containing all thirteen values. `add_student()` then checks whether the roll number already exists; if it is new, the dictionary is converted into a one-row DataFrame, appended to the existing data and written back to the CSV file. A success dialog appears and the form is cleared.

8.3 Update Student

An existing record can be modified in two ways. Clicking a row of the table calls `fill_form_from_tree(event)`, which copies the ten editable values of that row into the form. The values can then be edited and saved with Update. Alternatively, the roll number can be typed directly into the Roll Number field. The update function locates the row with `df.loc[df["Roll Number"] == roll, ...]` and replaces all thirteen values of that row; if the roll number does not exist in the file, an error dialog reports that the roll number was not found.

8.4 Delete Student

Deletion works with the roll number present in the Roll Number field. The program first verifies that the roll number exists, then asks for confirmation with a yes/no dialog before removing anything. On confirmation, the record is removed by keeping every row whose roll number is different from the one entered, the file is saved again, and a success dialog is displayed. If the field contains a non-numeric value, an error dialog asks for a valid roll number instead.

```

207 def delete_student():
208     try:
209         roll = int(roll_var.get().strip())
210         df = pd.read_csv(FILE_NAME)
211
212         if roll not in df["Roll Number"].values:
213             messagebox.showerror("Error", "Roll Number not found!")
214             return
215
216         confirm = messagebox.askyesno("Confirm Delete", "Are you sure you want to delete this student?")
217         if confirm:
218             df = df[df["Roll Number"] != roll]
219             df.to_csv(FILE_NAME, index=False)
220             messagebox.showinfo("Success", "Student Deleted Successfully!")
221             clear_form()
222             refresh_table()
223     except ValueError:
224         messagebox.showerror("Error", "Please enter a valid Roll Number to delete.")
225
226

```

Source: [Project Program.py](#) — deletion with roll number validation and a confirmation dialog.

8.5 Import CSV

The Import CSV button opens the standard file chooser filtered to CSV files. The selected file is read with Pandas and passed through four checks before a single row is accepted: all thirteen required columns must be present, the numeric columns are converted with `pd.to_numeric(..., errors="coerce")`, the file must contain no missing or invalid value, and none of its roll numbers may already exist in the current data file. Only when all four checks pass are the imported rows appended to the existing records and saved. The workflow is shown in detail in Chapter 13.

8.6 Search

The search box accepts either a name or a roll number. The query is converted to lower case and matched against the Roll Number column (converted to text) and the Name column, so a partial entry also works. Only the matching records are displayed in the table, and the KPI cards are recalculated for the displayed subset. Searching for a name such as `reddy` therefore shows every student whose name contains that text, and entering a roll number shows the matching record.

8.7 View All

The View All button clears the search box and calls `refresh_table()` without a filtered DataFrame, which reloads the complete data file. It is the standard way of returning to the full list after a search.

8.8 Automatic Calculations

Total marks, percentage and grade are computed inside the input routine, not typed by the user. This keeps the three derived columns consistent with the subject marks at all times, both for records added through the form and for records loaded through the CSV import. The grade thresholds are documented in Chapter 7.

8.9 Analytics

Ten analytics buttons are arranged in a grid below the record table. Every button calls `show_analytics(action)` with its own label; the function computes the requested summary from the current CSV data and displays the result in a message box titled with the name of the operation. If the file contains no records, an information dialog states that no data is available.

Analytics button	Pandas expression used	Result shown in the message box
Class Average	<code>df.groupby("Class")["Percentage"].mean()</code>	Average percentage of Class 11 and Class 12 separately
Topper	<code>df.loc[df["Percentage"].idxmax()]</code>	Name, percentage and class of the highest scorer
Lowest Scorer	<code>df.loc[df["Percentage"].idxmin()]</code>	Name, percentage and class of the lowest scorer
Top 5	<code>df.nlargest(5, "Percentage")</code>	Five students with the highest percentage
Bottom 5	<code>df.nsmallest(5, "Percentage")</code>	Five students with the lowest percentage
Subject Avg	<code>df[["English","Physics","Chemistry","Mathematics"]].mean()</code>	Average marks of each of the four subjects
Pass Count	<code>len(df[df["Grade"] != "F"])</code>	Number of students whose grade is not F
Fail Count	<code>len(df[df["Grade"] == "F"])</code>	Number of students with grade F
Pass Pct	<code>(pass_count / len(df)) * 100</code>	Pass percentage of the class
Grade Dist	<code>df["Grade"].value_counts()</code>	How many students received each grade

Table 8.2 — The ten analytics operations, the Pandas expression behind each one and the information displayed to the user.

8.10 Graphical Visualisation

Three chart buttons are placed at the bottom of the window. Each calls `plot_graph(graph_type)`, which reads the data file, creates a figure of size 8 × 5 inches and draws one of the following charts:

- **Top Students Chart** — a bar chart of the five students with the highest percentage, with a different colour for each bar and labelled axes.
- **Subject Average Chart** — a bar chart comparing the average marks of English, Physics, Chemistry and Mathematics.

- **Grade Distribution Pie** — a pie chart showing the share of each grade, with percentages printed on the slices.

The charts are displayed in a separate Matplotlib window, and the plot function returns immediately with an information dialog if the data file is empty.

```

346 def plot_graph(graph_type):
347     df = pd.read_csv(FILE_NAME)
348     if df.empty:
349         messagebox.showinfo("Info", "No data to plot.")
350         return
351
352     plt.figure(figsize=(8, 5))
353
354     if graph_type == "Top Students":
355         top_students = df.nlargest(5, "Percentage")
356         plt.bar(top_students["Name"], top_students["Percentage"], color=["blue", "green", "red",
"purple", "orange"])
357         plt.title("Top 5 Students by Percentage")
358         plt.xlabel("Student Name")
359         plt.ylabel("Percentage (%)")
360     elif graph_type == "Subject Average":
361         averages = df[["English", "Physics", "Chemistry", "Mathematics"]].mean()
362         plt.bar(averages.index, averages.values, color=["skyblue", "lightgreen", "salmon", "plum"])
363         plt.title("Class Average in Each Subject")
364         plt.xlabel("Subjects")
365         plt.ylabel("Average Marks")
366     elif graph_type == "Grade Distribution":
367         grade_counts = df["Grade"].value_counts()
368         plt.pie(grade_counts.values, labels=grade_counts.index, autopct="%1.1f%%", startangle=140)
369         plt.title("Grade Distribution of Students")
370
371     plt.tight_layout()
372     plt.show()
373
374
375 # =====
376 # GUI LAYOUT DESIGN
377 # =====
378

```

Source: `Project Program.py` — the complete chart function containing all three chart types.

CHAPTER 09

User Interface Design

Layout, widgets, colours and typography of the application window

The whole application lives in one window of 1400 × 800 pixels, styled with the CustomTkinter “Dark” appearance mode and the “blue” colour theme. The layout is divided into a header, a dashboard strip, a left-hand form panel and a right-hand data area.

●●● Student Management System — 1400 × 800 window, dark appearance

STUDENT MANAGEMENT SYSTEM

CBSE Informatics Practices Project

Total Students

20

Average Percentage

77.44%

Topper Name

Sneha Reddy

Pass Percentage

95.00%

Student Details Form

Roll Number:

101

Student Name:

Aarav Sharma

Class:

12 ▾

Section:

A ▾

Gender:

Male ▾

Age:

17

English Marks:

85

Physics Marks:

92

Chemistry Marks:

88

Maths Marks:

90

Add

Update

Delete

Import CSV

Clear Fields

Search by Name or Roll Number

Search

View All

Roll Number	Name	Class	Section	Gender	Age	English	Physics	Chemistry
101	Aarav Sharma	12	A	Male	17	85	92	88
102	Priya Patel	12	A	Female	17	78	85	82
103	Rohan Gupta	12	B	Male	18	92	95	90
104	Anjali Singh	12	B	Female	17	70	65	72
105	Vikram Mehta	11	A	Male	16	88	91	89

: 20 records loaded · vertical scrollbar for rows, horizontal scrollbar to reach Mathematics, Total Marks, Percentage and Grade

Data Analytics & Reports

Class Average

Topper

Lowest Scorer

Top 5

Bottom 5

Subject Avg

Pass Count

Fail Count

Pass Pct

Grade Dist

Top Students Chart

Subject Average Chart

Grade Distribution Pie

Status · students.csv loaded · 20 records read · dashboard recalculated · waiting for the next action

Figure 9.1 — Illustrative interface layout based on the submitted source code. The record values shown are taken from the supplied sample dataset.

Labelling: this is an illustrative reconstruction of the interface drawn from the widget names, sizes, colours and text strings found in the source code. It is not a screen capture of the running program.

9.1 Screen regions

Region	How it is built in the code
Application header	Two centred labels — STUDENT MANAGEMENT SYSTEM in bold 32-point Arial and the subtitle CBSE Informatics Practices Project — inside a transparent frame.
KPI row	Four cards built by <code>create_kpi_card()</code> ; each card is a rounded #2b2b2b frame with the caption in 14-point text and the value in bold 24-point text coloured #1f6aa5.
Student details form	A 350-pixel-wide panel titled Student Details Form holding ten controls created by <code>make_entry()</code> . Class, Section and Gender are combo boxes; the rest are text entries.
Record buttons	Add, Update and Delete on the first row (Delete in red), Import CSV in green on the same row, and a full-width grey Clear Fields button on the second row.
Search area	A 300-pixel entry with the placeholder Search by Name or Roll Number, followed by the Search and View All buttons.
Record table	A <code>ttk.Treeview</code> with all thirteen columns, 90-pixel column width, centred values, dark background #2b2b2b, heading background #1f1f1f, 25-pixel row height and both scrollbars.
Analytics grid	Ten buttons of 110 pixels width in the colours #3a3a3a / #4a4a4a, arranged five per row by the grid loop.
Chart buttons	Three buttons of 180 pixels width in the accent colour #1f6aa5.

Table 9.1 — Interface regions and the code that produces them.

9.2 Colour and typography

The interface uses only the colours declared in the source code together with the defaults of the CustomTkinter dark theme. The values that the program sets explicitly are listed below.

Element	Colour / value set in the source
Window appearance	<code>ctk.set_appearance_mode("Dark")</code> — dark theme for all widgets
Accent colour theme	<code>ctk.set_default_color_theme("blue")</code>
KPI card background	#2b2b2b with corner radius 10
KPI value text	#1f6aa5, bold 24-point Arial
Analytics buttons	#3a3a3a, hover #4a4a4a
Chart buttons	#1f6aa5
Import CSV button	#2e7d32, hover #1b5e20
Delete button	red, hover darkred
Clear Fields / View All	gray
Table background	#2b2b2b, text white, row height 25
Table heading	#1f1f1f, white bold 10-point text

Element	Colour / value set in the source
Selected row	#1f6aa5
Headline type	Arial 32 bold for the title, Arial 16 for the subtitle, Arial 20 bold for the form title and Arial 16 bold for the analytics label

Table 9.2 — Colour and typography values set explicitly by the program. All remaining colours are supplied by the CustomTkinter dark theme.

9.3 Colour palette used by the program

Every colour below is taken from the source code. The remaining colours of the window are supplied by the CustomTkinter dark appearance theme.

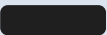
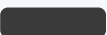
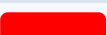
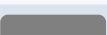
Sample	Value	Used for
	#1f6aa5	KPI values, chart buttons, selected table row
	#2b2b2b	KPI card frames and table background
	#1f1f1f	Table column headings
	#3a3a3a	Analytics buttons (hover #4a4a4a)
	#2e7d32	Import CSV button (hover #1b5e20)
	red	Delete button (hover darkred)
	gray	Clear Fields and View All buttons

Table 9.3 — colour values written explicitly in the source code.

```

389 def create_kpi_card(parent, title):
390     card = ctk.CTkFrame(parent, fg_color="#2b2b2b", corner_radius=10)
391     card.pack(side="left", fill="both", expand=True, padx=10, pady=10)
392     ctk.CTkLabel(card, text=title, font=("Arial", 14)).pack(pady=(10, 0))
393     val_label = ctk.CTkLabel(card, text="-", font=("Arial", 24, "bold"), text_color="#1f6aa5")
394     val_label.pack(pady=(5, 10))
395     return val_label
396
397

```

Source: [Project Program.py](#) — the function that draws one KPI card, showing the fixed colours and font sizes of the dashboard.

KEY POINT — WHY CUSTOMTKINTER FOR THE INTERFACE?

CustomTkinter keeps the simplicity of Tkinter while providing rounded widgets, consistent spacing and a complete dark theme. One call — `set_appearance_mode("Dark")` — restyles every widget in the window, which is why the interface remains visually consistent with very little code.

CHAPTER 10

Program Logic

The functions of the program and how the application starts

The program is organised as a small set of functions, each responsible for one operation. The table below documents every function of the source file, the job it performs and the key operation inside it. Line numbers refer to the complete listing in Appendix A.

Function	Purpose	Key operation
<code>calculate_grade(percentage)</code>	Convert a percentage into a grade.	Compares the value with 90, 80, 70, 60 and 40 in order and returns A+, A, B, C, D or F.
<code>clear_form()</code>	Reset the input form.	Sets every form variable back to an empty string; Class, Section and Gender return to the default values 12, A and Male.
<code>refresh_table(search_df=None)</code>	Redraw the record table.	Deletes all rows of the Treeview, loads the data file with Pandas (or uses a supplied filtered DataFrame), inserts one row per student and calls <code>update_dashboard(df)</code> .
<code>update_dashboard(df)</code>	Recalculate the four KPI cards.	Computes the record count, mean percentage, topper name and pass percentage; shows 0 / 0.0% / N/A / 0.0% when the DataFrame is empty.
<code>get_form_data()</code>	Collect and validate the form values.	Checks that no field is blank, converts roll number, age and marks to numbers, validates their ranges, computes total, percentage and grade, and returns a dictionary of thirteen values.
<code>add_student()</code>	Insert a new student record.	Reads the CSV file, rejects a roll number that already exists, appends the new row with <code>pd.concat()</code> , saves the file, clears the form and refreshes the table.
<code>update_student()</code>	Modify an existing record.	Verifies the roll number, replaces all thirteen values of that row with <code>df.loc[]</code> , saves the file and refreshes the table.
<code>delete_student()</code>	Remove a record.	Reads the roll number from the form, verifies it, asks for confirmation and keeps only the rows whose roll number differs from the entered one before saving.
<code>import_csv()</code>	Load records from an external CSV file.	Opens a file chooser, checks the required columns, selects the expected columns, converts numeric fields, rejects missing or invalid data and duplicate roll numbers, then appends and saves.
<code>search_student()</code>	Filter the table by name or roll number.	Lower-cases the query, matches it against the text form of Roll Number and the Name column, and refreshes the table with the filtered DataFrame.
<code>fill_form_from_tree(event)</code>	Copy a selected row into the form.	Reads the ten editable values of the selected Treeview row and places them in the form variables so the record can be edited.

Function	Purpose	Key operation
<code>show_analytics(action)</code>	Compute one analytical summary.	Reads the CSV file, evaluates the Pandas expression for the requested action and displays the result in a message box titled with the action name.
<code>plot_graph(graph_type)</code>	Draw one of the three charts.	Reads the CSV file, creates an 8 × 5 inch figure and draws the top-five bar chart, the subject-average bar chart or the grade-distribution pie chart before calling <code>plt.show()</code> .
<code>create_kpi_card(parent, title)</code>	Build one dashboard card.	Creates a rounded #2b2b2b frame containing the caption label and a bold value label in #1f6aa5, and returns the value label for later updates.
<code>make_entry(parent, label, variable, is_combo, combo_values)</code>	Build one form row.	Creates a frame with a fixed-width label on the left and either a <code>CTKEntry</code> or a <code>CTKComboBox</code> bound to the supplied <code>StringVar</code> on the right.

Table 10.1 — Every function of the submitted source file, its purpose and its principal operation. The functions are listed in the order in which they appear in the program.

10.1 Program flow from start-up to first screen



Figure 10.1 — Start-up sequence of the application, from module execution to the event loop.

```

500 | analytics_list = [
501 |     "Class Average",

```

```

502     "Topper",
503     "Lowest Scorer",
504     "Top 5",
505     "Bottom 5",
506     "Subject Avg",
507     "Pass Count",

```

Source: [Project Program.py](#) — the end of the program: the initial call to `refresh_table()` and the start of the Tkinter event loop.

10.2 Function-to-feature map

Feature of the application	Functions involved
Start-up	module level code, <code>refresh_table()</code>
Add a student	<code>get_form_data()</code> , <code>calculate_grade()</code> , <code>add_student()</code> , <code>clear_form()</code> , <code>refresh_table()</code>
Update a student	<code>fill_form_from_tree()</code> , <code>get_form_data()</code> , <code>update_student()</code> , <code>refresh_table()</code>
Delete a student	<code>delete_student()</code> , <code>clear_form()</code> , <code>refresh_table()</code>
Import a file	<code>import_csv()</code> , <code>refresh_table()</code> , <code>update_dashboard()</code>
Search and reset	<code>search_student()</code> , <code>refresh_table()</code>
Analytics	<code>show_analytics()</code>
Charts	<code>plot_graph()</code>
Interface construction	<code>create_kpi_card()</code> , <code>make_entry()</code> , widget calls

Table 10.2 — the functions that participate in each feature of the application.

KEY POINT — WHAT IS THE EVENT LOOP?

`app.mainloop()` starts Tkinter's event loop. From that point the program does not run from top to bottom; it waits for an event such as a button click, and each event calls the function linked to that button. This is why the file ends with the event loop while all the real work happens inside the functions.

CHAPTER 11

Input Validation

Checks that protect the data file from invalid records

Validation is performed before any value is written to the data file. Every check below exists in the submitted source code; the messages are the actual strings passed to the Tkinter message boxes.

Check	What is verified	How it is implemented	Message shown
Required fields	All ten form fields must contain something	The values are collected in a list and tested together	Warning — "All fields are required!"
Roll number type	Roll number must be a whole number	int(roll) inside a try / except ValueError block	Error — "Roll Number, Age and Marks must be valid numbers!"
Age type	Age must be a whole number	int(age) inside the same block	Error — the same message as above
Marks type	Each subject mark must be a number	float(eng), float(phy), float(chem), float(math)	Error — the same message as above
Age range	Age greater than 0 and not more than 100	if age <= 0 or age > 100	Error — "Please enter a valid age."
Marks range	Every subject mark between 0 and 100	for mark in [eng, phy, chem, math]: if mark < 0 or mark > 100	Error — "Marks must be between 0 and 100."
Duplicate roll number (form)	A roll number that already exists cannot be added again	if data["Roll Number"] in df["Roll Number"].values	Error — "Student with this Roll Number already exists!"
Missing roll number (update / delete)	The record to be changed must exist	if roll not in df["Roll Number"].values	Error — "Roll Number not found in database!" / "Roll Number not found!"
Delete input	The roll number field must hold a valid number	int(roll_var.get().strip()) inside try / except ValueError	Error — "Please enter a valid Roll Number to delete."
Import — missing columns	The imported file must contain all thirteen column headings	missing_cols = set(COLUMNS) - set(import_df.columns)	Error — "Missing required columns: ..."
Import — invalid values	Numeric columns are converted; empty or non-numeric entries are detected	pd.to_numeric(..., errors="coerce") followed by import_df.isnull().any().any()	Error — "CSV contains invalid or missing data in required fields."
Import — duplicate roll numbers	Imported roll numbers must not clash with existing records	set(import_df["Roll Number"]).intersection(set(existing_df["Roll Number"]))	Error — "Duplicate Roll Numbers found: ..."

Check	What is verified	How it is implemented	Message shown
Unreadable file	Any other failure while reading or writing the file is reported	try / except around the whole import operation	Error — “Failed to import CSV: ...”
Table refresh failure	Data that cannot be loaded is reported instead of crashing the program	try / except inside refresh_table()	Error — “Could not load data: ...”

Table 11.1 — Complete list of validations implemented in the project and the message displayed to the user in each case.

11.1 Validation sequence in the form



Figure 11.1 — Order of the validation checks performed by `get_form_data()` before a record reaches the data file.

```

111 def get_form_data():
112     try:
113         roll = roll_var.get().strip()
114         name = name_var.get().strip()
115         cls = class_var.get()
116         sec = section_var.get()
117         gen = gender_var.get()
118         age = age_var.get().strip()
119         eng = eng_var.get().strip()
120         phy = phy_var.get().strip()
121         chem = chem_var.get().strip()
122         math = math_var.get().strip()
123
124         if not all([roll, name, cls, sec, gen, age, eng, phy, chem, math]):

```

```

125         messagebox.showwarning("Warning", "All fields are required!")
126         return None
127
128         roll = int(roll)
129         age = int(age)
130         eng = float(eng)
131         phy = float(phy)
132         chem = float(chem)
133         math = float(math)
134
135         if age <= 0 or age > 100:
136             messagebox.showerror("Error", "Please enter a valid age.")
137             return None
138
139         for mark in [eng, phy, chem, math]:
140             if mark < 0 or mark > 100:
141                 messagebox.showerror("Error", "Marks must be between 0 and 100.")
142                 return None
143
144         total = eng + phy + chem + math
145         percentage = total / 4.0
146         grade = calculate_grade(percentage)
147
148         return {
149             "Roll Number": roll,
150             "Name": name,
151             "Class": cls,
152             "Section": sec,
153             "Gender": gen,
154             "Age": age,
155             "English": eng,
156             "Physics": phy,
157             "Chemistry": chem,
158             "Mathematics": math,
159             "Total Marks": total,
160             "Percentage": percentage,
161             "Grade": grade,
162         }
163     except ValueError:
164         messagebox.showerror("Error", "Roll Number, Age and Marks must be valid numbers!")
165         return None
166

```

Source: `Project Program.py` — the validation block: blank fields, numeric conversion, age range and marks range.

KEY POINT — HOW IS DUPLICATE DATA PREVENTED?

Twice. While adding a single record, the program checks `data["Roll Number"] in df["Roll Number"].values` before writing. While importing a file, it compares the set of imported roll numbers with the set of existing roll numbers and stops if the two sets overlap. A roll number therefore always identifies exactly one student.

KEY POINT — WHY IS VALIDATION DONE BEFORE SAVING?

Because the CSV file is the only copy of the data. If an invalid record were written first and rejected later, the file would already be corrupted. Validating in `get_form_data()` means the file is touched only after the record is known to be correct.

CHAPTER 12

CRUD Operations

Create, Read, Update and Delete in the implemented application

Create, Read, Update and Delete are the four fundamental operations on stored data. All four are implemented in this project through Pandas DataFrame operations on the CSV file, and all four are reachable from the buttons of the main window.

CREATE — add_student()

The validated dictionary is converted into a one-row DataFrame and appended with `pd.concat([df, pd.DataFrame([data]), ignore_index=True]`, then the whole DataFrame is written back with `df.to_csv(FILE_NAME, index=False)`.

READ — refresh_table() / search_student()

The file is loaded with `pd.read_csv(FILE_NAME)` and every row is inserted into the Treeview; a search passes a filtered DataFrame to the same function so only the matching rows are displayed.

UPDATE — update_student()

The target row is located with a boolean mask and all thirteen values are replaced at once: `df.loc[df["Roll Number"] == roll, list(data.keys())] = list(data.values())`. The file is then saved again.

DELETE — delete_student()

After confirmation, the record is removed by keeping every other row — `df = df[df["Roll Number"] != roll]` — and the reduced DataFrame is written back to the file.

Table 12.1 — The four CRUD operations and the Pandas expression used by each one.

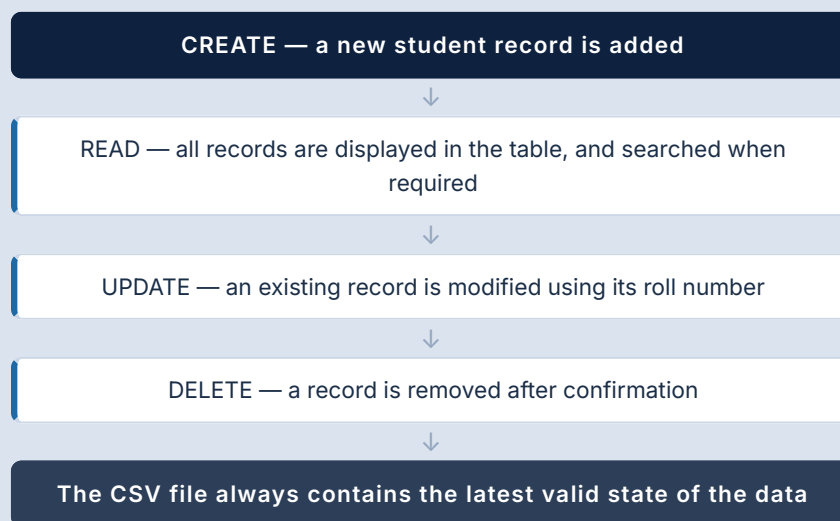


Figure 12.1 — CRUD life cycle as it occurs in the application.

12.1 How each operation reaches the file

Operation	Function	Data change	Confirmation shown
Create	<code>add_student()</code>	One row appended to the DataFrame, then the file is rewritten	"Student Added Successfully!"
Read	<code>refresh_table()</code>	No change — the file is read and displayed	—
Update	<code>update_student()</code>	Thirteen values of one row replaced, then the file is rewritten	"Student Updated Successfully!"

Operation	Function	Data change	Confirmation shown
Delete	<code>delete_student()</code>	The selected row removed, then the file is rewritten	Yes / No confirmation, then "Student Deleted Successfully!"

Table 12.2 — Each CRUD operation, its function and the change made to the stored data.

12.2 Read and write cycle of the data file

Every operation follows the same three steps, which is why the file always stays consistent:

1. **Load** — `pd.read_csv(FILE_NAME)` reads the complete file into a DataFrame.
2. **Modify** — the operation changes the DataFrame in memory (append, replace, filter).
3. **Save** — `df.to_csv(FILE_NAME, index=False)` writes the entire DataFrame back, so the file on disk always reflects the last successful operation.

```

173 def add_student():
174     data = get_form_data()
175     if data:
176         df = pd.read_csv(FILE_NAME)
177
178         if data["Roll Number"] in df["Roll Number"].values:
179             messagebox.showerror("Error", "Student with this Roll Number already exists!")
180             return
181
182         df = pd.concat([df, pd.DataFrame([data])], ignore_index=True)
183         df.to_csv(FILE_NAME, index=False)
184
185         messagebox.showinfo("Success", "Student Added Successfully!")
186         clear_form()
187         refresh_table()
188
189

```

Source: `Project Program.py` — the create operation, including the duplicate roll-number check.

```

190 def update_student():
191     data = get_form_data()
192     if data:
193         df = pd.read_csv(FILE_NAME)
194         roll = data["Roll Number"]
195
196         if roll not in df["Roll Number"].values:
197             messagebox.showerror("Error", "Roll Number not found in database!")
198             return
199
200         df.loc[df["Roll Number"] == roll, list(data.keys())] = list(data.values())
201         df.to_csv(FILE_NAME, index=False)
202
203         messagebox.showinfo("Success", "Student Updated Successfully!")
204         refresh_table()
205
206

```

Source: `Project Program.py` — the update operation. All thirteen columns of the matching row are replaced in a single assignment.

KEY POINT — WHY IS THE WHOLE FILE REWRITTEN?

A CSV file is a plain text file, not a database with indexed records, so a single line cannot be updated in place safely. The program therefore loads the complete file into a DataFrame, applies the change and writes the complete DataFrame back. For the size of data this application handles, this is simple and completely reliable.

CHAPTER 13

Data Import Workflow

From selecting a file to refreshing the interface

The import feature allows a prepared CSV file — such as the twenty-record `Student.csv` supplied with this project — to be loaded into the application without typing the records one by one.



Figure 13.1 — The ten stages of the CSV import workflow, exactly as implemented in `import_csv()`.

13.1 Stage-by-stage description

Stage	What the program does	Result if the check fails
1	Opens the file chooser filtered to CSV files; the function returns silently if the user cancels.	Nothing is imported.
2-3	Reads the file and compares its column headings with the thirteen required columns.	Error listing the missing columns.
4	Rearranges the imported data so that only the expected columns, in the expected order, are taken.	Only reached when the check passes.
5	Converts Roll Number, Age, the four marks, Total Marks and Percentage to numbers; any entry that cannot be converted becomes a missing value.	Values stay empty and are caught in the next stage.

Stage	What the program does	Result if the check fails
6	Rejects the file if any required value is missing after conversion.	Error about invalid or missing data.
7	Compares the imported roll numbers with those already in the data file.	Error listing the duplicate roll numbers.
8-9	Appends the imported rows below the existing rows and writes the combined DataFrame to the data file.	—
10	Reloads the table and recalculates the four dashboard indicators.	—

Table 13.1 — Description of each import stage and the failure behaviour of the program.

13.2 Behaviour on each failure

If a check fails, the import stops immediately with an error dialog and the existing data file is left untouched. The four failure situations and their messages are:

- **Missing columns** — “Missing required columns: ...” followed by the names of the columns that are absent.
- **Invalid or missing values** — “CSV contains invalid or missing data in required fields.”
- **Duplicate roll numbers** — “Duplicate Roll Numbers found: ...” listing the roll numbers that already exist in the data file.
- **Unreadable file** — “Failed to import CSV: ...” with the underlying error message.

```

227 def import_csv():
228     file_path = tk.filedialog.askopenfilename(
229         title="Select CSV File",
230         filetypes=[("CSV Files", "*.csv"), ("All Files", "*.*")]
231     )
232     if not file_path:
233         return
234
235     try:
236         import_df = pd.read_csv(file_path)
237
238         missing_cols = set(COLUMNS) - set(import_df.columns)
239         if missing_cols:
240             messagebox.showerror("Error", f"Missing required columns: {' '.join(missing_cols)}")
241             return
242
243         import_df = import_df[COLUMNS]
244
245         for col in ["Roll Number", "Age"]:
246             import_df[col] = pd.to_numeric(import_df[col], errors="coerce")
247         for col in ["English", "Physics", "Chemistry", "Mathematics", "Total Marks", "Percentage"]:
248             import_df[col] = pd.to_numeric(import_df[col], errors="coerce")
249
250         if import_df.isnull().any().any():
251             messagebox.showerror("Error", "CSV contains invalid or missing data in required fields.")
252             return
253
254         existing_df = pd.read_csv(FILE_NAME)
255         duplicate_rolls = set(import_df["Roll Number"]).intersection(set(existing_df["Roll Number"]))
256         if duplicate_rolls:
257             messagebox.showerror("Error", f"Duplicate Roll Numbers found: {' '.join(map(str, duplicate_rolls))}")
258             return
259
260         combined_df = pd.concat([existing_df, import_df], ignore_index=True)
261         combined_df.to_csv(FILE_NAME, index=False)
262
263         messagebox.showinfo("Success", f"Imported {len(import_df)} students successfully!")
264         refresh_table()
265     except Exception as e:

```

```

266     messagebox.showerror("Error", f"Failed to import CSV: {e}")
267
268

```

Source: `Project Program.py` — the complete import function with all four validation stages.

13.3 Import checklist

#	Condition the imported file must satisfy	Where it is checked
1	All thirteen column headings are present, spelled as in the COLUMNS list	missing_cols check
2	The columns may be in any order — the program re-selects them	<code>import_df = import_df[COLUMNS]</code>
3	Roll Number and Age contain whole numbers	<code>pd.to_numeric(..., errors="coerce")</code>
4	The four subjects, Total Marks and Percentage contain numbers	<code>pd.to_numeric(..., errors="coerce")</code>
5	No required value is empty after conversion	<code>import_df.isnull().any().any()</code>
6	No imported roll number already exists in the data file	set intersection of roll numbers

Table 13.2 — the conditions checked before an imported file is accepted.

VIVA POINT — QUESTIONS ON THIS CHAPTER

Q. What happens if the imported file has the columns in a different order?

The import succeeds. The program selects the columns by name with `import_df = import_df[COLUMNS]` before any other check, so the order in the file does not matter as long as all thirteen headings exist.

Q. Why are the imported rows appended instead of replacing the file?

The existing records are read into `existing_df` and combined with the imported rows using `pd.concat()`. The data already stored in the application is therefore never lost during an import.

Q. What happens to the file if the import fails midway?

Nothing is written. Every check runs before the write step, so a failed import leaves the existing data file exactly as it was.

KEY POINT — HOW DOES THE CSV IMPORT WORK?

The imported file is treated as untrusted input. The program checks the column structure first, then the values, then the uniqueness of the roll numbers; only a file that passes all three checks is appended to the existing records. The supplied dataset `Student.csv` satisfies all these checks and is therefore ready to import without any modification.

CHAPTER 14

Search Workflow

How records are located by name or roll number

Search is performed entirely in memory. The CSV file is read once, the query is matched against two columns, and the filtered DataFrame is handed to the same table-refresh function that displays all records.

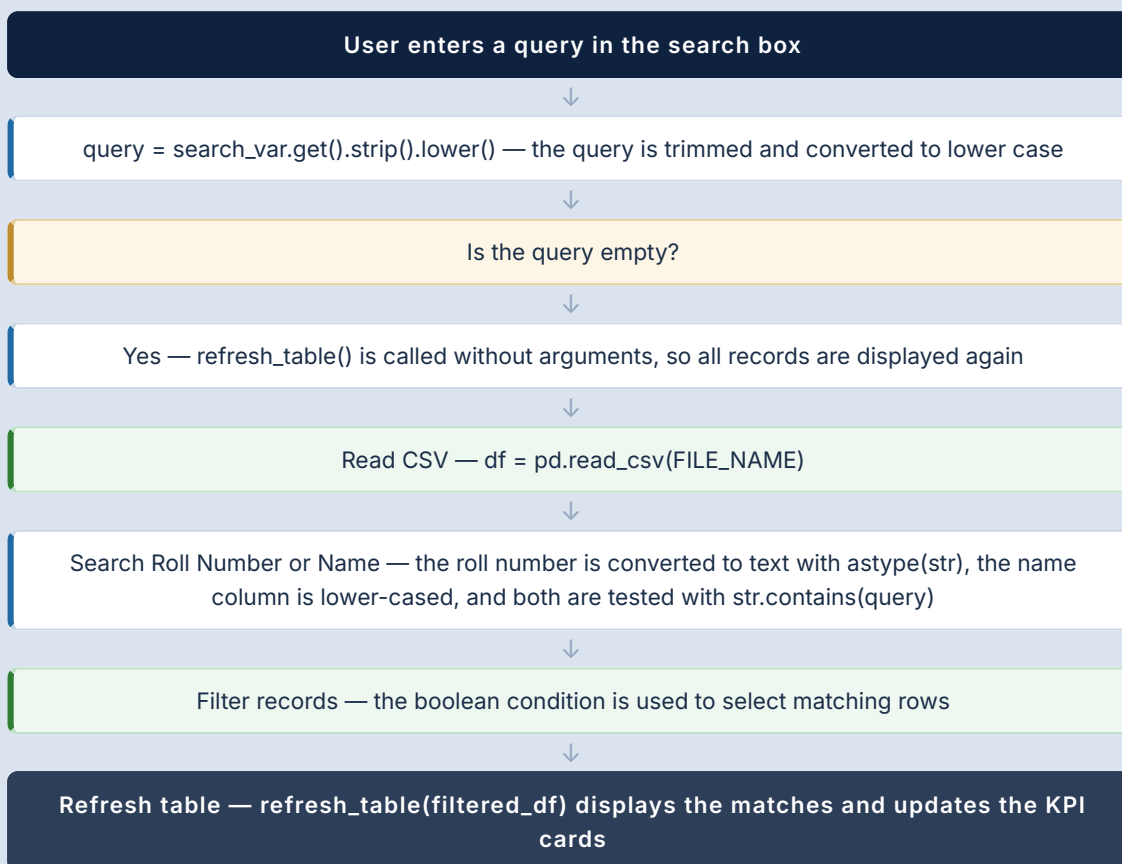


Figure 14.1 — Search workflow of search_student(), from the typed query to the refreshed table.

```

269 def search_student():
270     query = search_var.get().strip().lower()
271     if not query:
272         refresh_table()
273         return
274
275     df = pd.read_csv(FILE_NAME)
276     filtered_df = df[
277         df["Roll Number"].astype(str).str.contains(query) |
278         df["Name"].str.lower().str.contains(query)
279     ]
280     refresh_table(filtered_df)
281
282
  
```

Source: [Project Program.py](#) — the search function. The vertical bar between the two conditions is the logical OR operator.

Listing 14.1 — The complete search function — only six statements long.

Query entered	What the program matches	Records displayed (sample data)
reddy	Name column contains "reddy" (lower-case comparison)	1

Query entered	What the program matches	Records displayed (sample data)
ar	Name or Roll Number contains the text "ar"	multiple records
10	Roll Number converted to text contains "10"	10 records — 101 to 110 ...
1	Any roll number containing the digit 1	11 records
(blank)	No filter — <code>refresh_table()</code> shows the whole file	20

Table 14.1 — Behaviour of the search for typical queries. Because the roll number is compared as text, a partial number also produces a match.

VIVA POINT — QUESTIONS ON THIS CHAPTER

Q. Which two columns does the search examine?

Roll Number (converted to text with `astype(str)`) and Name (converted to lower case). The two conditions are joined by the logical OR operator, so a record is displayed if either column matches.

Q. Does the search modify the data file?

No. The search only reads the file and passes a filtered DataFrame to `refresh_table()`. No write operation is performed, so searching can never change a record.

Q. Why does a partial roll number such as 10 also show records 101 to 109?

Because the roll number is compared as text with `str.contains()`, which looks for the entered text anywhere inside the value rather than for a complete match.

KEY POINT — WHY CONVERT THE QUERY TO LOWER CASE?

Python compares strings case-sensitively, so "Reddy" and "reddy" would otherwise be treated as different values. Lower-casing both the query and the Name column makes the search case-insensitive.

KEY POINT — WHAT HAPPENS TO THE DASHBOARD DURING A SEARCH?

The four KPI cards are recalculated for the records that are currently displayed, because `refresh_table()` calls `update_dashboard()` with the filtered DataFrame. Searching for one student therefore shows values for that student, and clicking View All restores the values for the complete dataset.

CHAPTER 15

Analytics Module

The ten analytical operations and how each one is computed

The analytics module turns the stored records into ten ready summaries. Every button of the Data Analytics & Reports panel calls the same function with a different action name, so the calculation is decided by a single `if / elif` chain.

Metric	Method used in the code	Computation	Output shown
Class Average	<code>df.groupby("Class")["Percentage"].mean()</code>	Mean percentage of each class	Class-wise average printed as a Series with the class value as the index
Topper	<code>df.loc[df["Percentage"].idxmax()]</code>	Row with the highest percentage	Name, percentage and class of the topper
Lowest Scorer	<code>df.loc[df["Percentage"].idxmin()]</code>	Row with the lowest percentage	Name, percentage and class of the lowest scorer
Top 5	<code>df.nlargest(5, "Percentage")</code>	Five highest percentage values	Table of Name, Class and Percentage
Bottom 5	<code>df.nsmallest(5, "Percentage")</code>	Five lowest percentage values	Table of Name, Class and Percentage
Subject Avg	<code>df[["English", "Physics", "Chemistry", "Mathematics"]].mean()</code>	Mean marks of the four subjects	One average value per subject
Pass Count	<code>len(df[df["Grade"] != "F"])</code>	Number of students whose grade is not F	"Total Passed: n"
Fail Count	<code>len(df[df["Grade"] == "F"])</code>	Number of students with grade F	"Total Failed: n"
Pass Pct	<code>(pass_count / len(df)) * 100</code>	Pass count as a percentage of all records	"Pass Percentage: n.nn%"
Grade Dist	<code>df["Grade"].value_counts()</code>	Frequency of each grade	Grade-wise count, highest frequency first

Table 15.1 — Analytics matrix — every option, the exact Pandas expression used, the computation performed and the result presented to the user.

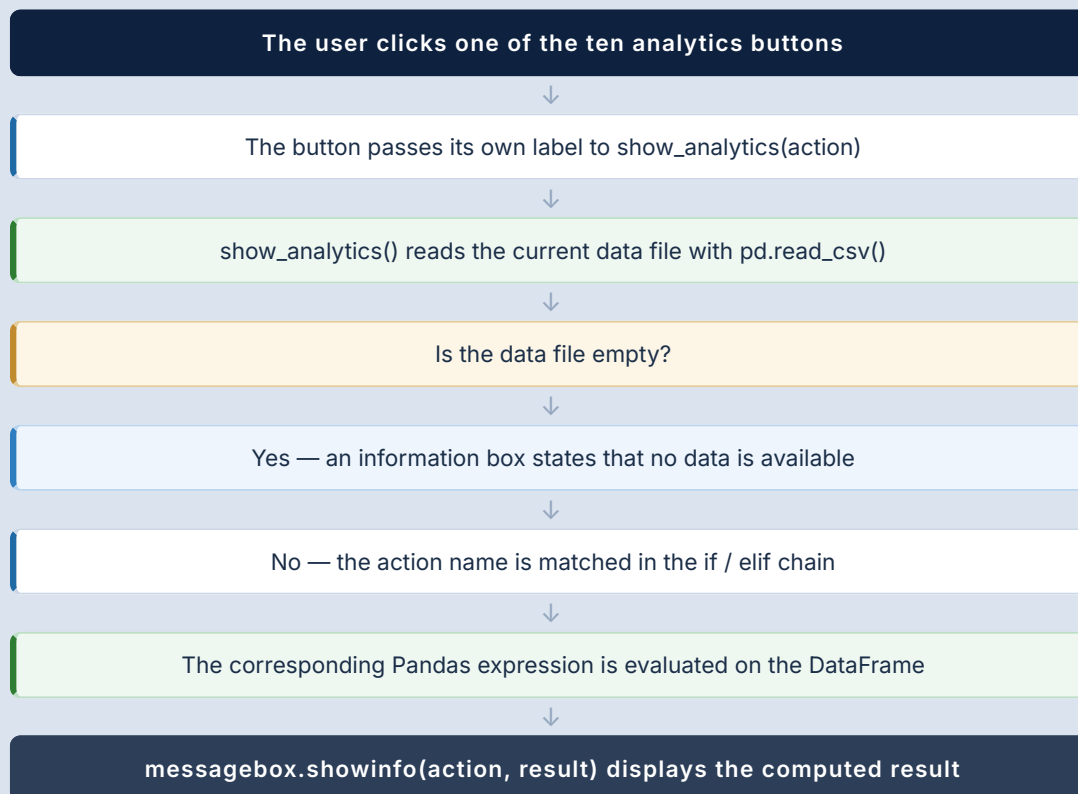


Figure 15.1 — The analytics pipeline — from the button clicked to the result on screen.

15.1 Expected output of each action on the sample dataset

Button	Result displayed for the twenty sample records
Class	11 → 81.125 12 → 73.750
Average	
Topper	Name: Sneha Reddy / Percentage: 94.5% / Class: 11
Lowest Scorer	Name: Nikhil Bansal / Percentage: 38.75% / Class: 12
Top 5	Sneha Reddy 94.50, Rohan Gupta 92.50, Riya Malhotra 92.50, Siddharth Rao 90.75, Neha Agarwal 89.75
Bottom 5	Nikhil Bansal 38.75, Arjun Kumar 48.75, Karan Verma 58.75, Rahul Shah 58.75, Tanvi Desai 67.50
Subject Avg	English 77.15 / Physics 77.05 / Chemistry 78.65 / Mathematics 76.90
Pass Count	Total Passed: 19
Fail Count	Total Failed: 1
Pass Pct	Pass Percentage: 95.00%
Grade Dist	A 8, A+ 4, D 3, C 2, B 2, F 1

Table 15.2 — the ten analytics results computed from the supplied sample dataset, used as the expected output during the practical examination.

Listing 15.1 — All ten analytics branches. Each branch prepares a formatted string that is displayed in a single message box.

```

305 def show_analytics(action):
306     df = pd.read_csv(FILE_NAME)
307     if df.empty:
308         messagebox.showinfo("Info", "No data available.")
309     return
  
```

```

310
311     result = ""
312     if action == "Class Average":
313         result = df.groupby("Class")["Percentage"].mean().to_string()
314     elif action == "Topper":
315         topper = df.loc[df["Percentage"].idxmax()]
316         result = f"Name: {topper['Name']}\nPercentage: {topper['Percentage']}%\nClass:
{topper['Class']}"
317     elif action == "Lowest Scorer":
318         lowest = df.loc[df["Percentage"].idxmin()]
319         result = f"Name: {lowest['Name']}\nPercentage: {lowest['Percentage']}%\nClass:
{lowest['Class']}"
320     elif action == "Top 5":
321         top5 = df.nlargest(5, "Percentage")[["Name", "Class", "Percentage"]]
322         result = top5.to_string(index=False)
323     elif action == "Bottom 5":
324         bot5 = df.nsmallest(5, "Percentage")[["Name", "Class", "Percentage"]]
325         result = bot5.to_string(index=False)
326     elif action == "Subject Avg":
327         subs = df[["English", "Physics", "Chemistry", "Mathematics"]].mean()
328         result = subs.to_string()
329     elif action == "Pass Count":
330         count = len(df[df["Grade"] != "F"])
331         result = f"Total Passed: {count}"
332     elif action == "Fail Count":
333         count = len(df[df["Grade"] == "F"])
334         result = f"Total Failed: {count}"
335     elif action == "Pass Pct":
336         count = len(df[df["Grade"] != "F"])
337         pct = (count / len(df)) * 100
338         result = f"Pass Percentage: {pct:.2f}%"
339     elif action == "Grade Dist":
340         dist = df["Grade"].value_counts()
341         result = dist.to_string()
342
343     messagebox.showinfo(action, result)
344
345

```

Source: [Project Program.py](#) — the complete analytics function with all ten actions.

KEY POINT — WHY PANDAS FOR THE ANALYTICS?

Each summary is a single expression — `mean()`, `groupby()`, `nlargest()`, `nsmallest()` or `value_counts()`. Replacing those with manual loops would take many lines each and would be far easier to get wrong. Pandas also handles the ordering automatically — for example Top 5 is sorted by percentage in descending order without any explicit sort call.

KEY POINT — HOW ARE PASS AND FAIL DECIDED?

By the grade stored in the file: every record whose Grade is not F counts as a pass, and every record with Grade F counts as a fail. Since the grade is itself derived from the percentage, the pass and fail counts follow the grade thresholds documented in Chapter 7.

CHAPTER 16

Data Visualisation

The three Matplotlib charts generated from the data

Matplotlib converts three of the analytical summaries into charts. All three are created by `plot_graph(graph_type)` from the records currently stored in the data file.

Note on the charts in this chapter — the three graphs below were produced by running the exact plotting code of the project (identical chart type, colours, titles and axis labels) on the supplied twenty-record dataset `Student.csv`. In the running application the same code opens the figure in a Matplotlib window through `plt.show()`.

16.1 Top Students Bar Chart

The chart takes the five students with the highest percentage using `df.nlargest(5, "Percentage")` and draws one bar per student, each in a different colour, with the title `Top 5 Students by Percentage` and the axes labelled `Student Name` and `Percentage (%)`.

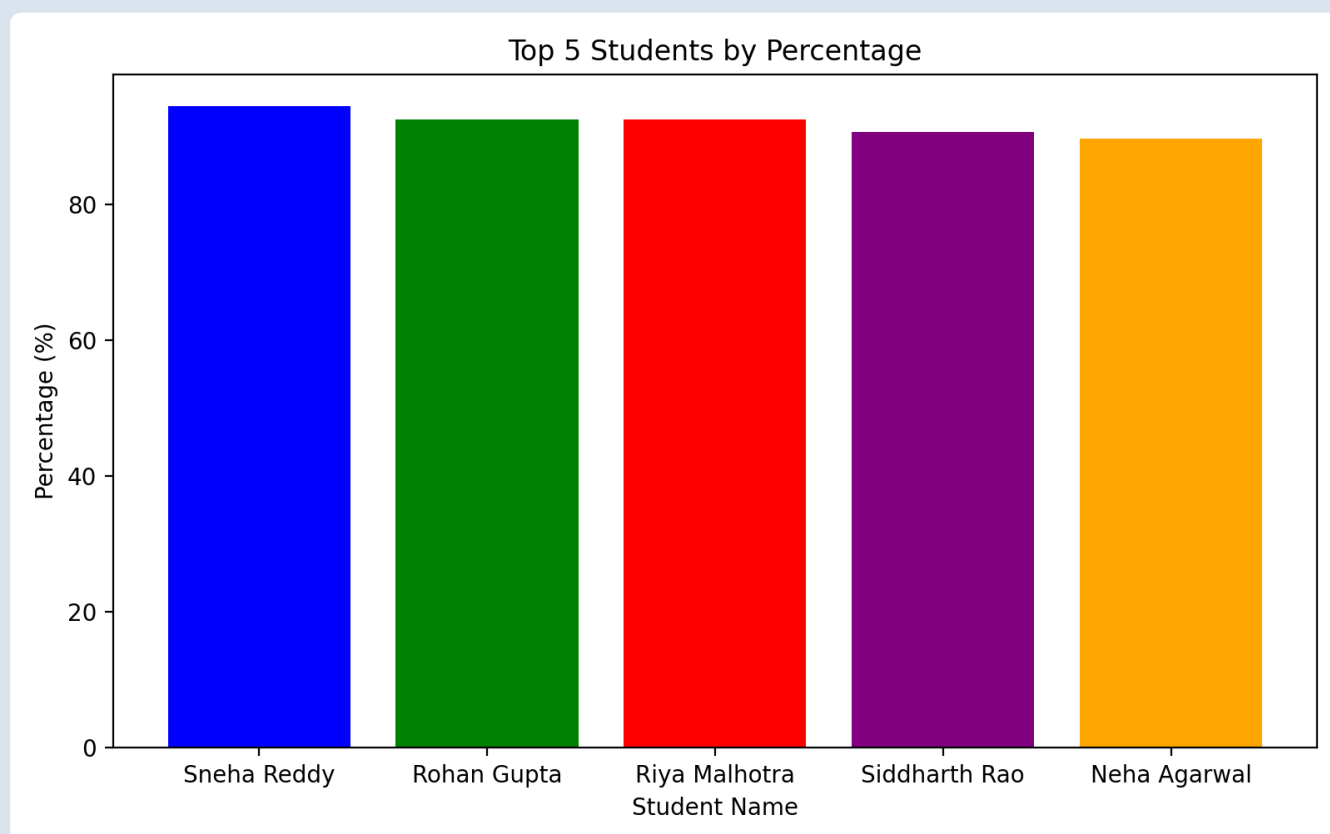


Figure 16.1 — Top five students by percentage, drawn from the supplied sample dataset.

16.2 Subject Average Bar Chart

The mean of each of the four subject columns is calculated with `df[["English", "Physics", "Chemistry", "Mathematics"]].mean()` and drawn as four bars with the title `Class Average in Each Subject`.

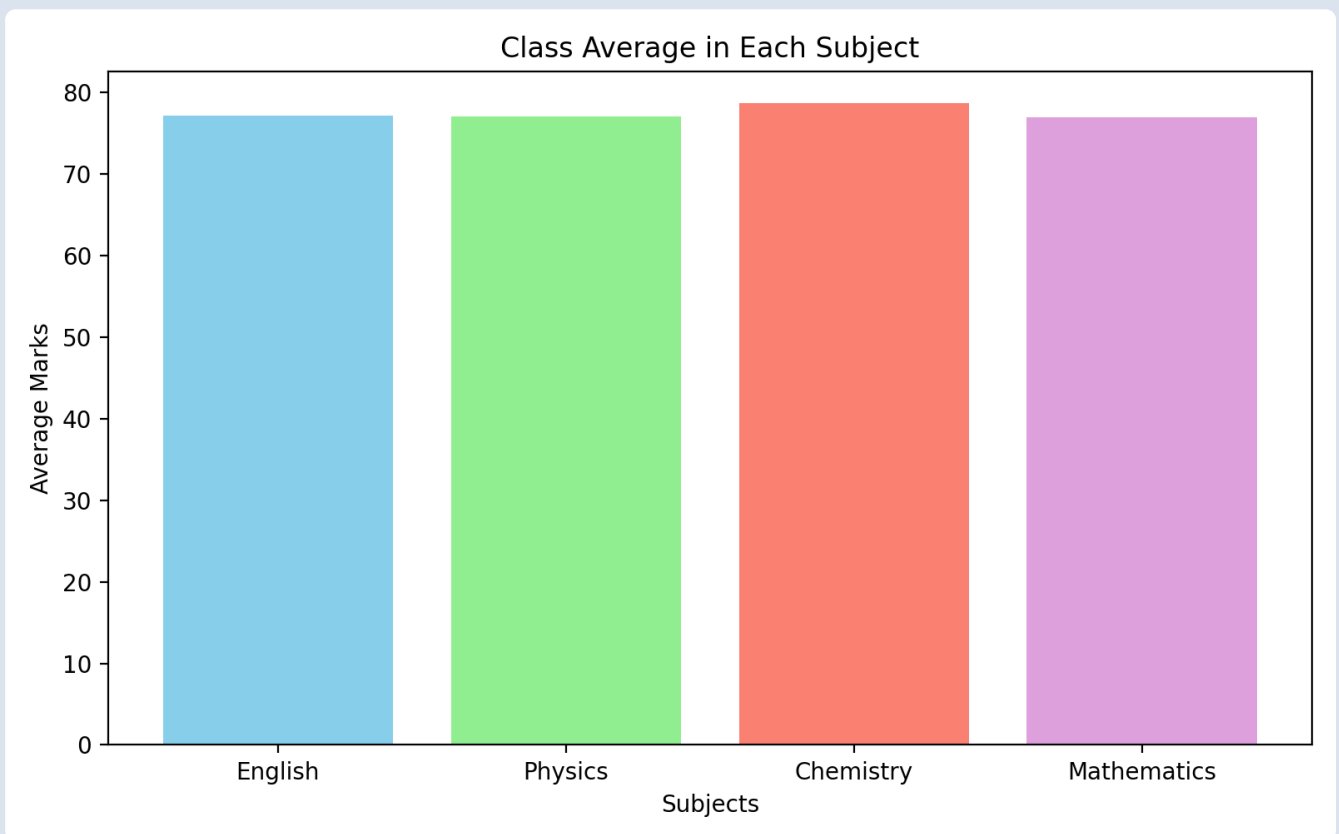


Figure 16.2 — Subject-wise average marks of the sample dataset. Chemistry has the highest average and Mathematics the lowest, with a difference of 1.75 marks.

16.3 Grade Distribution Pie Chart

The frequency of each grade is obtained with `df["Grade"].value_counts()` and drawn as a pie chart, with the percentage of each slice printed by `autopct="%1.1f%%"` and the first slice rotated to `startangle=140`.

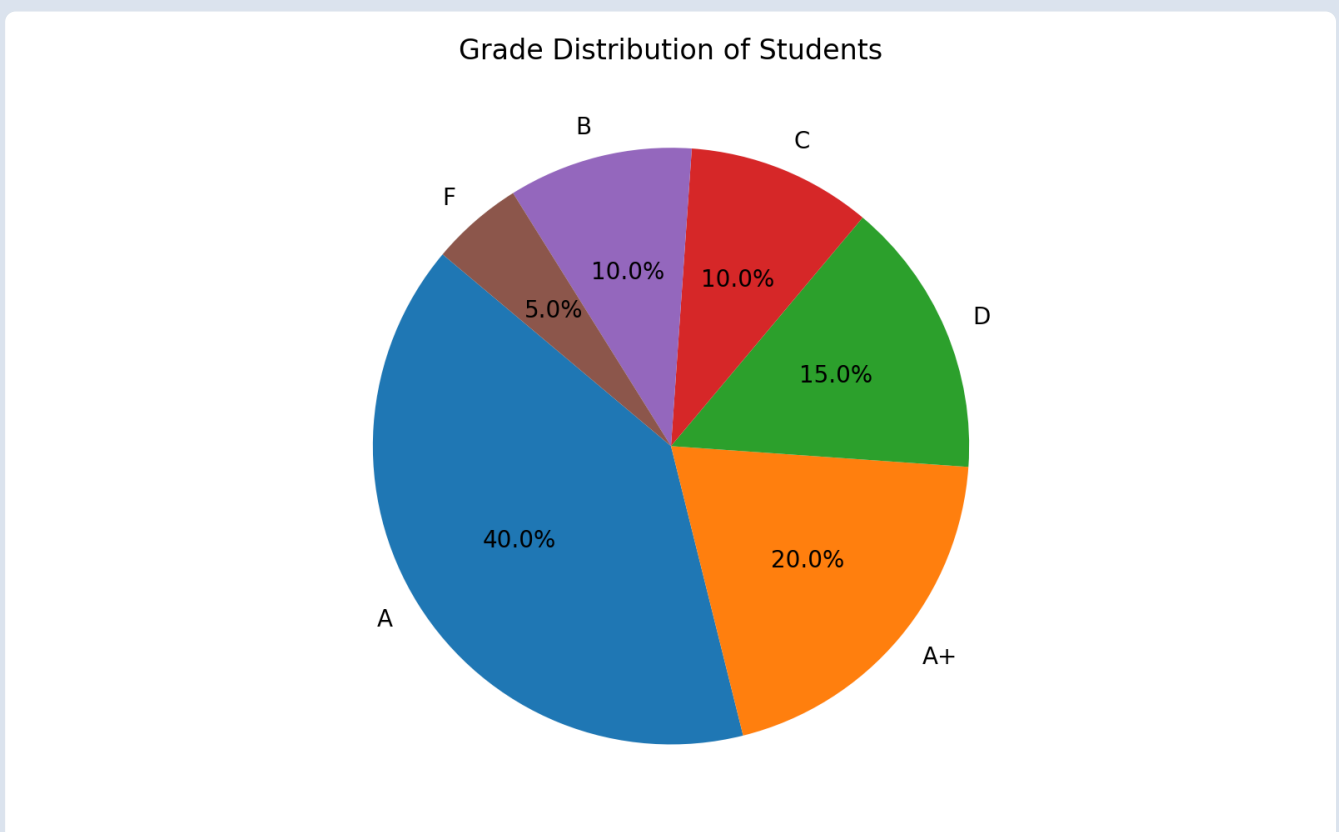


Figure 16.3 — Grade distribution of the sample dataset. The largest group is grade A with eight students, and one student is in grade F.

16.4 Reading the charts as a summary

Chart	What it shows	Reading of the sample dataset
Top Students Bar Chart	Comparison of the five best percentages	Sneha Reddy leads with 94.50%, followed by Rohan Gupta and Riya Malhotra, both at 92.50%.
Subject Average Bar Chart	Average performance in each subject	All four averages lie between 76.90 and 78.65, so the class performed almost equally in the four subjects.
Grade Distribution Pie Chart	Share of each grade in the class	Twelve of the twenty students (60%) are in grades A+ and A, while a single student (5%) is in grade F.

Table 16.1 — What each chart communicates about the sample dataset.

16.5 Presenting and saving a chart

Each chart opens in a separate Matplotlib window, which carries the standard toolbar at the bottom. The toolbar allows the figure to be zoomed, panned and saved as an image file, which is convenient when a chart has to be attached to a report or printed for a display board. The chart window is independent of the main application window, so the student table and the KPI cards remain visible while a chart is open.

Chart	Title set by the program	Axis labels
Top Students	Top 5 Students by Percentage	x: Student Name, y: Percentage (%)
Subject Average	Class Average in Each Subject	x: Subjects, y: Average Marks
Grade Distribution	Grade Distribution of Students	slice labels and percentages (autopct)

Table 16.2 — titles and labels assigned to the three charts.

VIVA POINT — QUESTIONS ON THIS CHAPTER

Q. Which chart type is used for which question?

Bar charts are used to compare values across categories — the five best percentages and the four subject averages. A pie chart is used to show how a whole is divided — the share of each grade.

Q. From which data are the charts drawn?

From the CSV file that is currently stored, read again each time a chart button is pressed. The charts therefore always match the records shown in the table.

Q. Why does the program check whether the data is empty before plotting?

An empty DataFrame would produce a chart without any bar or slice, and the pie chart would fail because there would be no values to represent. The check shows the message "No data to plot." instead.

KEY POINT — WHY MATPLOTLIB FOR THE VISUALISATION?

The same records that the table shows as numbers become comparable at a glance when they are drawn as bars or slices. Matplotlib performs this conversion with three or four statements per chart, and the resulting figure can be saved or printed directly from the chart window.

CHAPTER 17

Sample Dataset

Plug-and-play demonstration data supplied with the project

SAMPLE / DUMMY DATA — FOR DEMONSTRATION

The twenty records below are artificial test records supplied with the project. They do not represent real students of any school and are used only to demonstrate and test the application.

The dataset supplied with this project, `Student.csv`, contains twenty complete student records in exactly the thirteen-column format that the application expects. It is a plug-and-play dataset: it can be loaded through the Import CSV button without any modification, after which the table, the dashboard cards, the analytics and the charts are all populated immediately.

Roll No.	Name	Class	Section	Gender	Age	Total Marks
101	Aarav Sharma	12	A	Male	17	355
102	Priya Patel	12	A	Female	17	324
103	Rohan Gupta	12	B	Male	18	370
104	Anjali Singh	12	B	Female	17	275
105	Vikram Mehta	11	A	Male	16	355
106	Sneha Reddy	11	A	Female	16	378
107	Arjun Kumar	11	B	Male	17	195
108	Meera Joshi	11	B	Female	16	322
109	Karan Verma	12	C	Male	18	235
110	Neha Agarwal	12	C	Female	17	359
111	Aditya Nair	11	C	Male	16	299
112	Kavya Iyer	11	C	Female	16	350
113	Rahul Shah	12	D	Male	17	235
114	Pooja Menon	12	D	Female	17	330
115	Siddharth Rao	11	D	Male	16	363
116	Tanvi Desai	11	D	Female	16	270
117	Nikhil Bansal	12	A	Male	18	155
118	Divya Kapoor	12	A	Female	17	312
119	Manish Tiwari	11	B	Male	16	343
120	Riya Malhotra	11	B	Female	16	370

Table 17.1 — Part A of the sample dataset — identity and class details, with the calculated total marks.

Roll No.	Name	English	Physics	Chemistry	Mathematics	Percentage	Grade
101	Aarav Sharma	85	92	88	90	88.75	A
102	Priya Patel	78	85	82	79	81.00	A
103	Rohan Gupta	92	95	90	93	92.50	A+
104	Anjali Singh	70	65	72	68	68.75	C
105	Vikram Mehta	88	91	89	87	88.75	A
106	Sneha Reddy	95	93	96	94	94.50	A+
107	Arjun Kumar	45	50	48	52	48.75	D
108	Meera Joshi	82	79	81	80	80.50	A
109	Karan Verma	60	58	62	55	58.75	D
110	Neha Agarwal	90	88	92	89	89.75	A
111	Aditya Nair	75	72	78	74	74.75	B
112	Kavya Iyer	88	85	90	87	87.50	A
113	Rahul Shah	55	60	58	62	58.75	D
114	Pooja Menon	83	80	85	82	82.50	A
115	Siddharth Rao	91	89	93	90	90.75	A+
116	Tanvi Desai	68	70	65	67	67.50	C
117	Nikhil Bansal	40	38	42	35	38.75	F
118	Divya Kapoor	79	76	80	77	78.00	B
119	Manish Tiwari	86	84	88	85	85.75	A
120	Riya Malhotra	93	91	94	92	92.50	A+

Table 17.2 — Part B of the sample dataset — subject marks and the derived percentage and grade. The total marks of a record equal the sum of its four subject marks, and the percentage equals the total divided by four.

KEY POINT — WHY IS THE DATA SPLIT INTO TWO TABLES?

Thirteen columns cannot be printed legibly on a portrait A4 page. Instead of shrinking the font, the dataset is presented as two coordinated tables that share the roll-number column, and again as one complete landscape table in the original column order.

KEY POINT — COLUMN ORDER MATTERS

The application reads the data by column name, not by position; during import it re-selects the thirteen columns in the order defined in the `COLUMNS` constant. This is why the file supplied with this project can be imported directly, and also why a file with missing column headings is rejected with an error.

Continued overleaf — the same twenty records are also printed in landscape orientation in Table 17.3 on the next page, so that all thirteen columns can be read side by side in the original column order.

CHAPTER 17 — CONTINUED

Sample Dataset — Complete Table

All thirteen columns of Student.csv in the original column order · landscape orientation for readability

Roll Number	Name	Class	Section	Gender	Age	English	Physics	Chemistry	Mathematics	Total Marks	Percentage	Grade
101	Aarav Sharma	12	A	Male	17	85	92	88	90	355	88.75	A
102	Priya Patel	12	A	Female	17	78	85	82	79	324	81.00	A
103	Rohan Gupta	12	B	Male	18	92	95	90	93	370	92.50	A+
104	Anjali Singh	12	B	Female	17	70	65	72	68	275	68.75	C
105	Vikram Mehta	11	A	Male	16	88	91	89	87	355	88.75	A
106	Sneha Reddy	11	A	Female	16	95	93	96	94	378	94.50	A+
107	Arjun Kumar	11	B	Male	17	45	50	48	52	195	48.75	D
108	Meera Joshi	11	B	Female	16	82	79	81	80	322	80.50	A
109	Karan Verma	12	C	Male	18	60	58	62	55	235	58.75	D
110	Neha Agarwal	12	C	Female	17	90	88	92	89	359	89.75	A
111	Aditya Nair	11	C	Male	16	75	72	78	74	299	74.75	B
112	Kavya Iyer	11	C	Female	16	88	85	90	87	350	87.50	A
113	Rahul Shah	12	D	Male	17	55	60	58	62	235	58.75	D
114	Pooja Menon	12	D	Female	17	83	80	85	82	330	82.50	A
115	Siddharth Rao	11	D	Male	16	91	89	93	90	363	90.75	A+
116	Tanvi Desai	11	D	Female	16	68	70	65	67	270	67.50	C
117	Nikhil Bansal	12	A	Male	18	40	38	42	35	155	38.75	F
118	Divya Kapoor	12	A	Female	17	79	76	80	77	312	78.00	B
119	Manish Tiwari	11	B	Male	16	86	84	88	85	343	85.75	A
120	Riya Malhotra	11	B	Female	16	93	91	94	92	370	92.50	A+

Table 17.3 — SAMPLE / DUMMY DATA — FOR DEMONSTRATION. Complete contents of the supplied Student.csv, printed without any change to the values. This landscape page is part of the portrait report and continues the dataset begun on the previous page.

CHAPTER 18

Sample Analytical Results

Outputs computed from the supplied sample dataset

RESULTS GENERATED FROM THE SUPPLIED DUMMY DATASET

Every figure on this page was computed from the twenty records of `Student.csv` using the same Pandas operations that the application performs. They are demonstration outputs, not results of a real examination.

20

TOTAL STUDENTS

77.44%

AVERAGE PERCENTAGE

95.00%

PASS PERCENTAGE

94.50%

HIGHEST PERCENTAGE

Table 18.1 — Dashboard indicators of the sample dataset, corresponding to the four KPI cards of the application. The topper is **Sneha Reddy** with 94.50%, and the lowest scorer is **Nikhil Bansal** with 38.75%.

18.1 Class-wise average percentage

Produced by `df.groupby("Class")["Percentage"].mean()`. Class 11 records a higher average than Class 12 in this dataset.

Class	Average percentage	Exact value (before rounding)	Records
11	81.12%	81.125	10
12	73.75%	73.75	10

Table 18.2 — Mean percentage of each class in the sample dataset.

18.2 Subject-wise averages

Subject	Average marks	Equivalent percentage
English	77.15	77.2
Physics	77.05	77.0
Chemistry	78.65	78.7
Mathematics	76.90	76.9

Table 18.3 — Average marks of the four subjects across all twenty records.

18.3 Grade distribution

Grade	Number of students	Share of the class
A	8	40.0%
A+	4	20.0%
D	3	15.0%
C	2	10.0%
B	2	10.0%

Grade	Number of students	Share of the class
F	1	5.0%

Table 18.4 — Frequency of each grade, ordered from the most frequent to the least frequent, as returned by `value_counts()`.

18.4 Top five and bottom five students

#	Name	Class	Percentage	#	Name	Class	Percentage
1	Sneha Reddy	11	94.50%	1	Nikhil Bansal	12	38.75%
2	Rohan Gupta	12	92.50%	2	Arjun Kumar	11	48.75%
3	Riya Malhotra	11	92.50%	3	Karan Verma	12	58.75%
4	Siddharth Rao	11	90.75%	4	Rahul Shah	12	58.75%
5	Neha Agarwal	12	89.75%	5	Tanvi Desai	11	67.50%

Table 18.5 — Top five records (left) and bottom five records (right) of the sample dataset, as produced by `nlargest(5, "Percentage")` and `nsmllest(5, "Percentage")`.

```

91 def update_dashboard(df):
92     if df.empty:
93         kpi_total.configure(text="0")
94         kpi_avg.configure(text="0.0%")
95         kpi_topper.configure(text="N/A")
96         kpi_pass.configure(text="0.0%")
97         return
98
99     total_students = len(df)
100     avg_pct = df["Percentage"].mean()
101     topper_name = df.loc[df["Percentage"].idxmax()]["Name"]
102     pass_count = len(df[df["Grade"] != "F"])
103     pass_pct = (pass_count / total_students) * 100
104
105     kpi_total.configure(text=str(total_students))
106     kpi_avg.configure(text=f"{avg_pct:.2f}%")
107     kpi_topper.configure(text=str(topper_name))
108     kpi_pass.configure(text=f"{pass_pct:.2f}%")
109

```

Source: `Project Program.py` — the expressions that produce the four dashboard indicators listed above.

KEY POINT — HOW WERE THESE VALUES OBTAINED?

By loading the supplied dataset into a Pandas DataFrame and applying exactly the expressions used by the application — `mean()` for the averages, `groupby()` for the class-wise figures, `nlargest()` and `nsmllest()` for the extremes, and `value_counts()` for the grade distribution.

KEY POINT — EXPECTED ANALYTICS OUTPUT

Running the Class Average button on this dataset displays 11 → 81.125 and 12 → 73.750; the Pass Pct button displays Pass Percentage: 95.00%; and Grade Dist lists A: 8, A+: 4, D: 3, C: 2, B: 2, F: 1.

CHAPTER 19

Application Output

What the application displays for each main action

This chapter shows what the application displays for each of its main actions, using the supplied sample dataset. Every visual on the following pages is a reconstruction built from the widget definitions, colours and result strings found in the source code — it is not a screen capture of the running program, and it is labelled accordingly.

Illustrative output representation based on the submitted source code. No screenshot of the running program was available when this report was prepared, so the interface views in this chapter have been redrawn from the code. Record values, dialog text and dashboard figures are taken from the supplied sample dataset and from the exact expressions and message strings in the program.

19.1 Main Dashboard Output

After the sample dataset has been imported, the four dashboard cards display the values computed by `update_dashboard()`, and the table shows the records loaded from `students.csv`. Every value shown below corresponds to the calculations documented in Chapters 15 and 18.

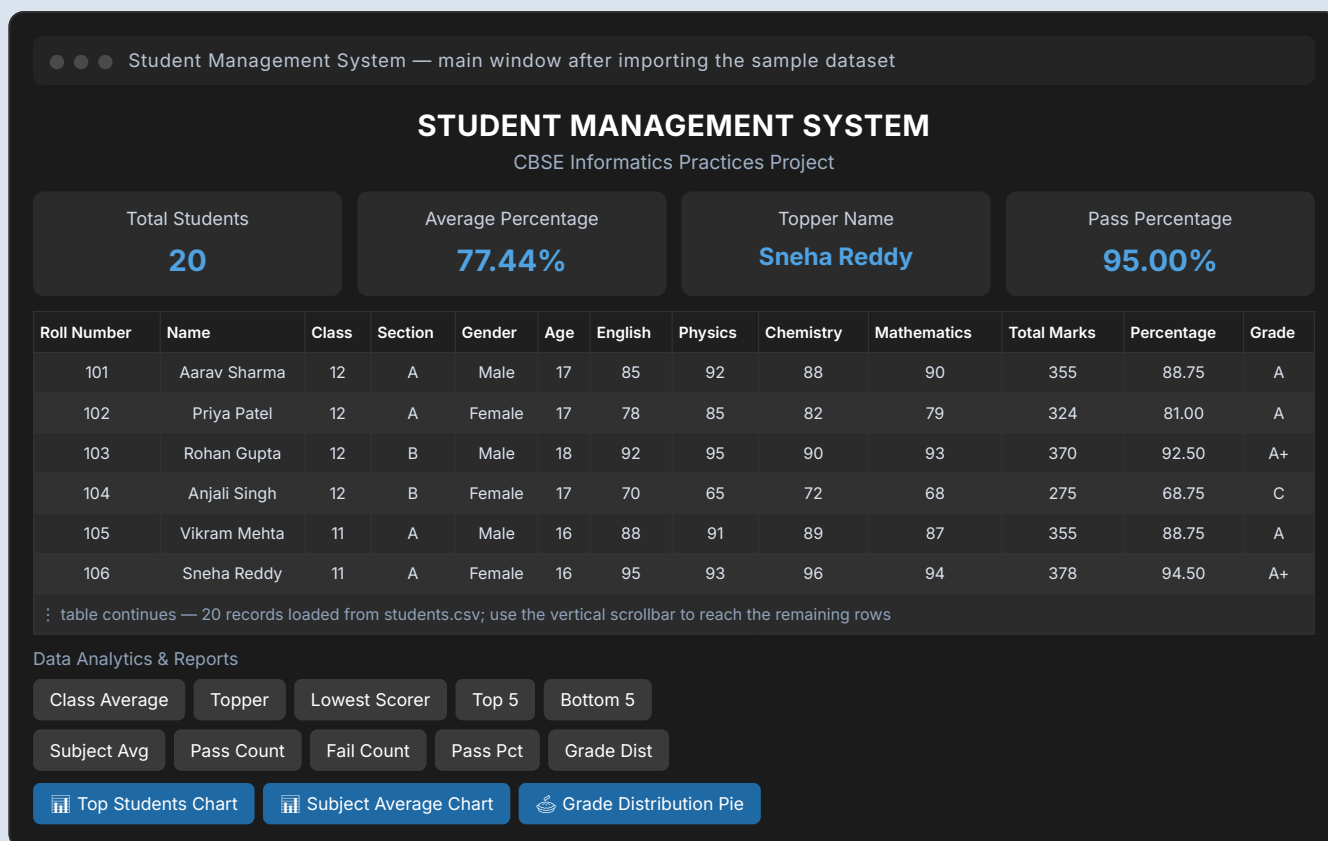


Figure 19.1 — Illustrative main-window output with the sample dataset loaded.

Reproduction note — in the running application the window is 1400 × 800 pixels and the record table shows all thirteen columns at once. This figure is scaled to the page width.

19.2 Populated Student Table

The table displays thirteen columns. Because all of them cannot be shown legibly on the width of this page, the figure below presents the left and right portions of the same rows; in the application the two portions appear as one continuous table that scrolls horizontally.

Student Management System — table area

Student Records — students.csv

20 records · 13 columns · vertical and horizontal scrollbars active

Left portion of the table (Roll Number → Age)

Roll Number	Name	Class	Section	Gender	Age
101	Aarav Sharma	12	A	Male	17
102	Priya Patel	12	A	Female	17
103	Rohan Gupta	12	B	Male	18
104	Anjali Singh	12	B	Female	17
105	Vikram Mehta	11	A	Male	16
106	Sneha Reddy	11	A	Female	16
107	Arjun Kumar	11	B	Male	17
108	Meera Joshi	11	B	Female	16

Right portion of the same rows, reached with the horizontal scrollbar (English → Grade)

English	Physics	Chemistry	Mathematics	Total Marks	Percentage	Grade
85	92	88	90	355	88.75	A
78	85	82	79	324	81.00	A
92	95	90	93	370	92.50	A+
70	65	72	68	275	68.75	C
88	91	89	87	355	88.75	A
95	93	96	94	378	94.50	A+
45	50	48	52	195	48.75	D
82	79	81	80	322	80.50	A

Figure 19.2 — Populated record table — the first eight of the twenty imported records. The last columns (Total Marks, Percentage, Grade) are reached with the horizontal scrollbar, as shown in the right-hand portion.

19.3 Search Result View

Entering the text `reddy` and clicking Search filters the table to the matching record. The dashboard cards are recalculated for the filtered DataFrame, so the indicators describe the displayed record and not the whole file. Clicking View All restores the complete table.

Student Management System — search result for the query “reddy”

reddy Search View All

Total Students 1	Average Percentage 94.50%	Topper Name Sneha Reddy	Pass Percentage 100.00%
----------------------------	-------------------------------------	-----------------------------------	-----------------------------------

Roll Number	Name	Class	Section	Gender	Age	English	Physics	Chemistry	Mathematics	Total Marks	Percentage	Grade
106	Sneha Reddy	11	A	Female	16	95	93	96	94	378	94.50	A+

Figure 19.3 — Search result view for the query “reddy”, including the recalculated dashboard cards.

19.4 Analytics Result Dialogs

Each analytics button opens an information box. The title of the box is the name of the operation and the body contains the computed result, exactly as produced by Pandas.

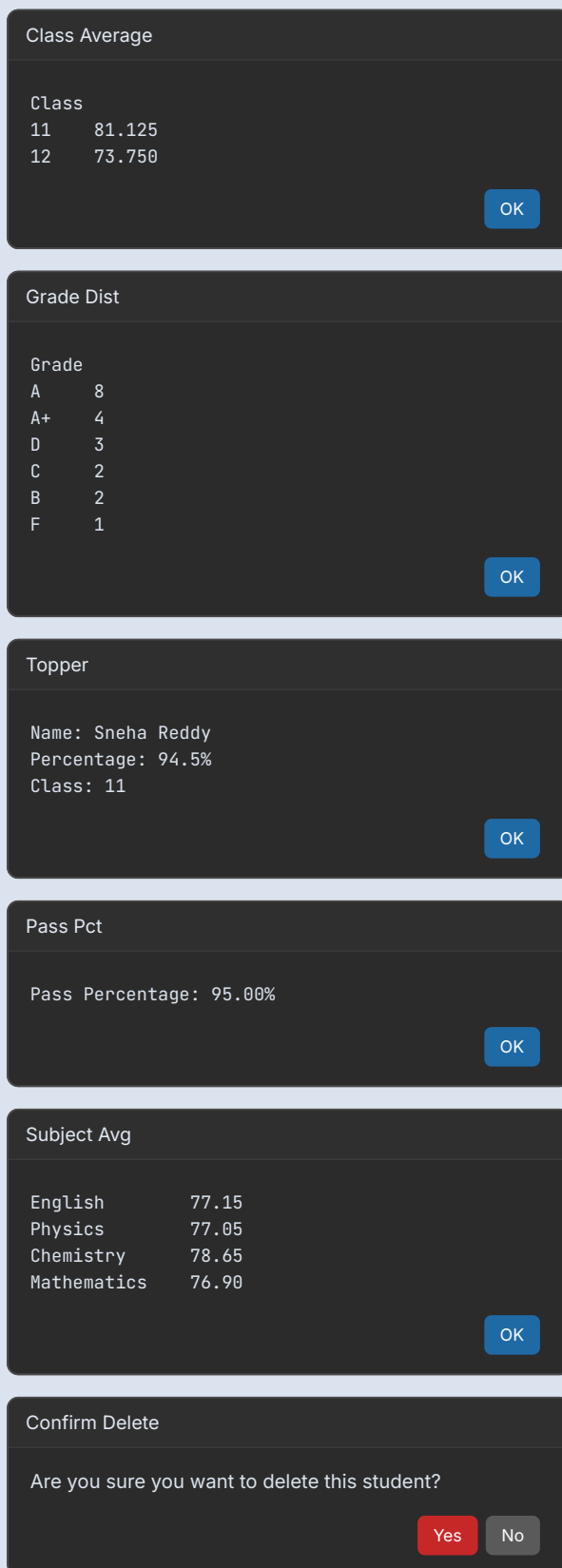


Figure 19.4 — Analytics dialogs produced from the sample dataset: class average, grade distribution, topper, pass percentage, subject averages and the delete confirmation dialog.

19.5 Chart Windows

The three chart buttons open a Matplotlib window containing one of the figures already presented in Chapter 16 — the top-five bar chart, the subject-average bar chart or the grade-distribution pie chart. In the running

application the figure appears in its own window with the standard Matplotlib toolbar, from which it can be saved as an image.

Chart button	Figure opened	Pandas data used
 Top Students Chart	Bar chart of the five best percentages (Figure 16.1)	<code>df.nlargest(5, "Percentage")</code>
 Subject Average Chart	Bar chart of the four subject averages (Figure 16.2)	<code>df[["English","Physics","Chemistry","Mathematics"]].mean()</code>
 Grade Distribution Pie	Pie chart of grade shares (Figure 16.3)	<code>df["Grade"].value_counts()</code>

Table 19.1 — The three chart buttons and the figure each one produces. The button captions are the exact strings used in the program.

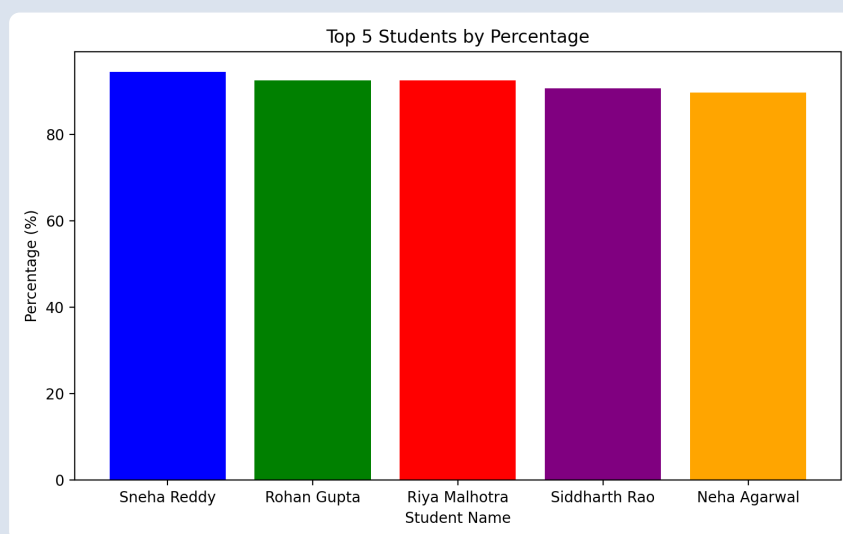


Figure 19.5 — The figure that appears in the Matplotlib window when  Top Students Chart is pressed, drawn from the sample dataset. It is the same figure presented in Chapter 16, shown here at the size at which it is placed in the report.

VIVA POINT — QUESTIONS ON THIS CHAPTER

Q. Why is the table shown in two halves?

The record table has thirteen columns, which is wider than a portrait A4 page can display legibly. The two halves in Figure 19.2 belong to the same rows; in the application they are one table reached with the horizontal scrollbar.

Q. Why do the dashboard cards change after a search?

Because `refresh_table()` passes whatever DataFrame it is given to `update_dashboard()`. The cards therefore always describe the records that are visible.

Q. Are the views in this chapter screenshots of the program?

No. No screenshot of the running program was available, so the interface views are reconstructions drawn from the widget definitions in the source code. The record values and the dialog text, however, are the actual values and strings produced by the program for the supplied sample dataset.

CHAPTER 20

Testing

Test cases derived from the implemented behaviour

The table below lists the test cases for the application. The expected results are taken from the validation logic and the message strings implemented in the source code, so each case can be reproduced from the running program during the practical examination or viva.

#	Test case	Input	Expected result	Implemented by
1	Add a valid student	All ten fields filled correctly, unused roll number	Record added; success dialog "Student Added Successfully!"; form cleared; table and KPI cards refreshed	get_form_data() → add_student()
2	Add a duplicate roll number	Roll number already present in the file	Error dialog "Student with this Roll Number already exists!"; the file is not modified	add_student() duplicate check
3	Leave required fields blank	One or more fields empty	Warning dialog "All fields are required!"; no record is saved	get_form_data() blank-field check
4	Enter invalid marks	Marks entered as 105, -5, or text such as "abc"	Error dialog "Marks must be between 0 and 100." or "Roll Number, Age and Marks must be valid numbers!"	get_form_data() range and conversion checks
5	Enter invalid age	Age entered as 0, 150 or a non-numeric value	Error dialog "Please enter a valid age." or the numeric-conversion error	get_form_data() age range check
6	Update an existing student	Existing roll number with corrected marks	Record updated; success dialog; recalculated total, percentage and grade; table refreshed	update_student() with df.loc assignment
7	Update a non-existing student	A roll number that is not in the file	Error dialog "Roll Number not found in database!"	update_student() existence check
8	Delete an existing student	Existing roll number, then "Yes" in the confirmation dialog	Confirmation asked first; record removed; success dialog; form cleared and table refreshed	delete_student() with messagebox.askyesno()
9	Search by student name	Search text such as "reddy"	Matching records displayed in the table; KPI cards recalculated for the filtered data	search_student() name condition
10	Search by roll number	Search text such as "103"	The matching record is displayed	search_student() roll-number condition
11	Import a valid CSV file	A file with all thirteen columns and valid values	Rows appended to the data file; success dialog "Imported n students successfully!"; table refreshed	import_csv() complete workflow
12	Import a CSV with missing columns	A file that does not contain all thirteen headings	Error dialog "Missing required columns: ..." listing the absent headings; no data imported	import_csv() column check

#	Test case	Input	Expected result	Implemented by
13	Import a CSV with duplicate roll numbers	A file whose roll numbers already exist	Error dialog "Duplicate Roll Numbers found: ..."; no data imported	import_csv() duplicate check
14	Display analytics	Any analytics button, for example Class Average	Information dialog named after the operation, containing the computed Pandas result	show_analytics() if / elif chain
15	Generate a chart with available data	Any chart button with records in the file	Matplotlib window opens with the corresponding bar or pie chart	plot_graph() with plt.show()
16	Generate a chart with an empty file	All records deleted, then a chart button pressed	Information dialog "No data to plot." instead of an empty or broken figure	plot_graph() empty-data guard
17	Analytics with an empty file	All records deleted, then an analytics button pressed	Information dialog "No data available."	show_analytics() empty-data guard
18	Cancel the import file chooser	Close the file dialog without selecting a file	Nothing happens; no error dialog is shown	import_csv() early return on empty file path

Table 20.1 — Test cases for the application with the expected behaviour of the submitted implementation.

KEY POINT — WHY TEST THE INVALID INPUTS FIRST?

The validity of a record system depends on what it refuses to store. Cases 2 to 5, 7 and 12 to 13 check that the program rejects a duplicate roll number, a blank field, an out-of-range mark and an invalid file before anything is written to the CSV file.

KEY POINT — HOW TO REPEAT THESE TESTS

Start the program, import the supplied Student.csv so that twenty records are available, and then perform each case from the interface. The data file can be restored at any time by deleting the extra records or by re-importing the sample dataset into an empty file.

CHAPTER 21

Error Handling

How the program reacts to invalid input and failures

Error handling in this project has one purpose: no invalid input or unreadable file should ever close the application or leave the data file in an incorrect state. The program uses three standard mechanisms — a try / except block, inline validation, and Tkinter message boxes.

21.1 try / except blocks

Four sections of the program are protected by exception handling:

Protected function	Block used	What it handles
<code>get_form_data()</code>	try / except ValueError	Wraps the conversion of roll number, age and the four marks. If any value cannot be converted to a number, the error dialog "Roll Number, Age and Marks must be valid numbers!" is shown and the function returns None, so no operation is performed.
<code>delete_student()</code>	try / except ValueError	Wraps the reading of the roll number from the form. A non-numeric entry produces the dialog "Please enter a valid Roll Number to delete."
<code>import_csv()</code>	try / except Exception	Wraps the complete import operation — reading the chosen file, checking columns, converting values and writing the combined data. Any unexpected failure is reported as "Failed to import CSV: ..." with the message of the original error.
<code>refresh_table()</code>	try / except Exception	Wraps the reading of the data file and the filling of the table. If the file cannot be read, the dialog "Could not load data: ..." is displayed instead of the program closing unexpectedly.

Table 21.1 — The four exception-handling blocks of the program and the situation each one covers.

```

73 def refresh_table(search_df=None):
74     for row in tree.get_children():
75         tree.delete(row)
76
77     try:
78         if search_df is not None:
79             df = search_df
80         else:
81             df = pd.read_csv(FILE_NAME)
82
83         for _, row in df.iterrows():
84             tree.insert("", "end", values=list(row))
85
86         update_dashboard(df)
87     except Exception as e:
88         messagebox.showerror("Error", f"Could not load data: {e}")
89
90

```

Source: [Project Program.py](#) — exception handling in `refresh_table()`. The error message of the exception is included in the dialog, which helps locate the cause of a failure.

21.2 Message boxes

Dialog	Used for	Examples from the source
<code>messagebox.showinfo</code>	Confirming a successful operation or reporting an informational result	"Student Added Successfully!", "No data available.", the analytics results
<code>messagebox.showwarning</code>	Warning that stops the operation but is caused by incomplete input	"All fields are required!"
<code>messagebox.showerror</code>	Reporting rejected input or a failed operation	"Marks must be between 0 and 100.", "Roll Number not found in database!", "Failed to import CSV ..."
<code>messagebox.askyesno</code>	Asking for confirmation before an irreversible action	"Are you sure you want to delete this student?"

Table 21.2 — The four kinds of message box used by the application and the situations in which each one appears.

21.3 What the program does not do

The implementation has no logging framework, no custom exception classes and no error file. Errors are reported directly to the user through the interface and are not recorded anywhere else, which is appropriate for a single-user desktop application of this size.

KEY POINT — WHY IS A CONFIRMATION DIALOG IMPORTANT?

Deleting a record removes it from the CSV file permanently — there is no undo, because each operation rewrites the whole file. The yes/no dialog therefore provides the operator with one final opportunity to cancel the action, which is the simplest reliable safeguard against accidental data loss in this design.

CHAPTER 22

Algorithm / Logical Flow

The complete logic of the application as flowcharts

The complete behaviour of the application can be read as three flows: the start-up and main loop of the program, the processing of a single student record, and the reporting path used by the analytics and charts.

22.1 Main program flow



Figure 22.1 — Main program flow, from start-up to the closing of the application.

22.2 Student record processing

Every record that enters the system — whether typed into the form or loaded from an imported file — passes through the same processing path.

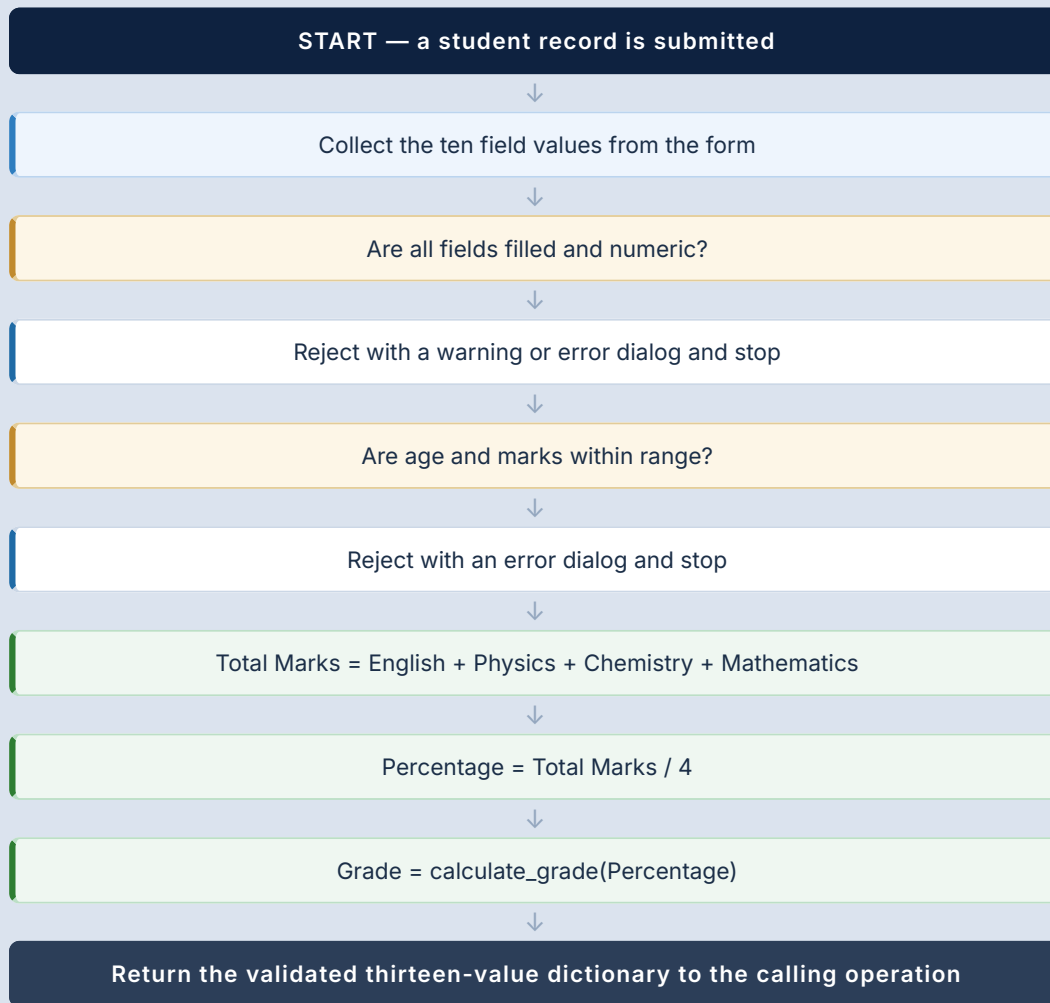


Figure 22.2 — Processing path of one student record, showing the derived values and the two exit points of validation.

22.3 Reporting path

The analytics and chart buttons never modify the data. They read the current file, compute the requested result and display it, which is why the same data can be summarised in ten different ways without any risk to the records.

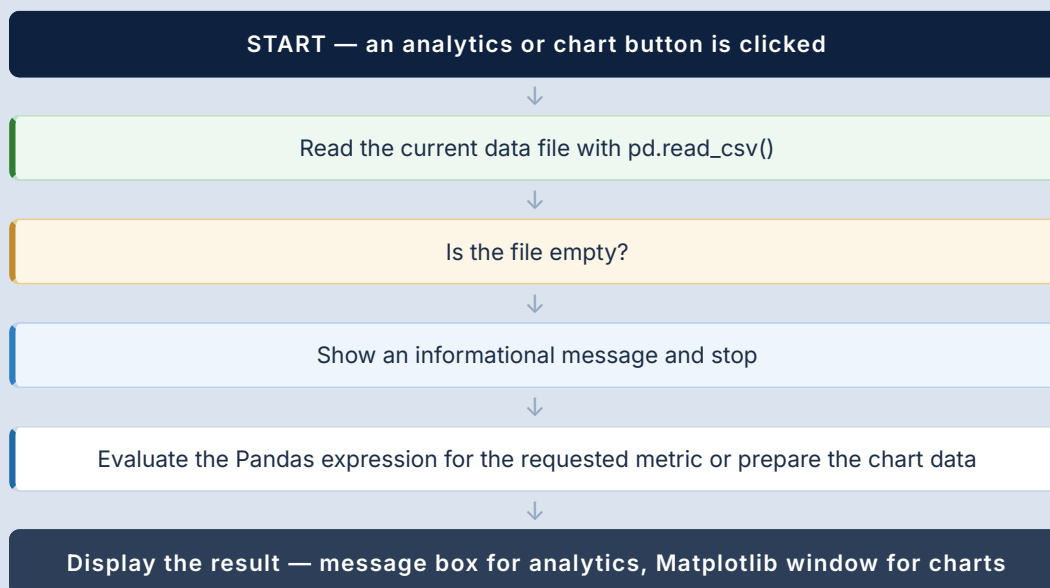


Figure 22.3 — Reporting path shared by the ten analytics operations and the three charts.

22.4 Flowchart notation used in this chapter

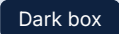
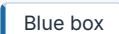
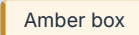
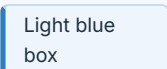
Shape	Meaning	Example from the charts above
	Start or end of the flow	START, END, "Repeat until the user closes the window"
	Process or operation performed by the program	"Convert numeric fields", "Calculate the grade"
	Decision that has more than one outcome	"Does the data file exist?", "Is the file empty?"
	Interaction with the user	"Wait for a user action", "Show an informational message"
	Data operation	"Write df back to the data file", "Load data into a DataFrame"

Table 22.1 — notation used in the flowcharts of this chapter.

VIVA POINT — QUESTIONS ON THIS CHAPTER

Q. Where does the program wait for the user?

At `app.mainloop()`. Tkinter runs its event loop there and calls the function linked to a widget whenever that widget is used; the program itself does not loop over the interface.

Q. Which flow is followed when the CSV import fails?

The reporting path is not used at all. The import function stops at the failing check, displays the error dialog and returns without writing to the data file, so neither the table nor the dashboard changes.

KEY POINT — READING THE FLOWS TOGETHER

Flow 1 describes the life of the program, flow 2 the life of one record and flow 3 the life of one report. The three flows together account for every function in the source file, which is a convenient way to revise the project before a viva.

CHAPTER 23

Pseudocode

Language-independent logic of the main operations

The pseudocode below describes the main operations of the application in plain structured language. It corresponds directly to the implemented functions listed in Chapter 10 and to the complete source listing in Appendix A.

23.1 Add Student

```
FUNCTION add_student():  
    data ← get_form_data()           # validates and computes total, percentage, grade  
    IF data is None THEN  
        RETURN                       # the validation step already showed the message  
    END IF  
    df ← READ the data file into a DataFrame  
    IF data["Roll Number"] EXISTS in df["Roll Number"] THEN  
        SHOW ERROR "Student with this Roll Number already exists!"  
        RETURN  
    END IF  
    df ← APPEND the new record to df  
    WRITE df back to the data file  
    SHOW INFO "Student Added Successfully!"  
    CALL clear_form()  
    CALL refresh_table()  
END FUNCTION
```

Pseudocode 23.1 — add_student()

23.2 Update Student

```
FUNCTION update_student():  
    data ← get_form_data()  
    IF data is None THEN RETURN END IF  
    df ← READ the data file  
    roll ← data["Roll Number"]  
    IF roll DOES NOT EXIST in df["Roll Number"] THEN  
        SHOW ERROR "Roll Number not found in database!"  
        RETURN  
    END IF  
    REPLACE all thirteen values of the row where Roll Number = roll  
    WRITE df back to the data file  
    SHOW INFO "Student Updated Successfully!"  
    CALL refresh_table()  
END FUNCTION
```

Pseudocode 23.2 — update_student()

23.3 Delete Student

```

FUNCTION delete_student():
    TRY
        roll ← CONVERT the Roll Number field to an integer
    EXCEPT
        SHOW ERROR "Please enter a valid Roll Number to delete."
        RETURN
    END TRY
    df ← READ the data file
    IF roll DOES NOT EXIST in df["Roll Number"] THEN
        SHOW ERROR "Roll Number not found!"
        RETURN
    END IF
    answer ← ASK YES/NO "Are you sure you want to delete this student?"
    IF answer = YES THEN
        df ← KEEP every row whose Roll Number is NOT equal to roll
        WRITE df back to the data file
        SHOW INFO "Student Deleted Successfully!"
        CALL clear_form()
        CALL refresh_table()
    END IF
END FUNCTION

```

Pseudocode 23.3 — delete_student()

23.4 Import CSV

```

FUNCTION import_csv():
    file_path ← OPEN FILE CHOOSER filtered to *.csv
    IF file_path is empty THEN RETURN END IF
    TRY
        import_df ← READ the chosen file
        missing ← REQUIRED COLUMNS - COLUMNS PRESENT in import_df
        IF missing is not empty THEN
            SHOW ERROR "Missing required columns: " + missing
            RETURN
        END IF
        import_df ← KEEP only the thirteen expected columns, in order
        CONVERT Roll Number and Age to numbers, invalid entries becoming empty
        CONVERT the four subjects, Total Marks and Percentage to numbers likewise
        IF ANY required value is empty THEN
            SHOW ERROR "CSV contains invalid or missing data in required fields."
            RETURN
        END IF
        existing_df ← READ the data file
        duplicates ← INTERSECTION of imported and existing Roll Numbers
        IF duplicates is not empty THEN
            SHOW ERROR "Duplicate Roll Numbers found: " + duplicates
            RETURN
        END IF
        combined ← APPEND import_df below existing_df
        WRITE combined back to the data file
        SHOW INFO "Imported " + COUNT(import_df) + " students successfully!"
        CALL refresh_table()
    EXCEPT error
        SHOW ERROR "Failed to import CSV: " + error
    END TRY
END FUNCTION

```

Pseudocode 23.4 — import_csv()

23.5 Search Student

```

FUNCTION search_student():
    query ← LOWERCASE(TRIM(the search box text))
    IF query is empty THEN
        CALL refresh_table()          # display all records again
        RETURN
    END IF
    df ← READ the data file
    filtered ← ROWS WHERE
        (Roll Number as text) CONTAINS query
        OR (LOWERCASE(Name)) CONTAINS query
    CALL refresh_table(filtered)
END FUNCTION

```

Pseudocode 23.5 — search_student()

23.6 Calculate Grade

```

FUNCTION calculate_grade(percentage):
    IF percentage >= 90 THEN RETURN "A+"
    ELSE IF percentage >= 80 THEN RETURN "A"
    ELSE IF percentage >= 70 THEN RETURN "B"
    ELSE IF percentage >= 60 THEN RETURN "C"
    ELSE IF percentage >= 40 THEN RETURN "D"
    ELSE RETURN "F"
END FUNCTION

FUNCTION compute_derived_values(english, physics, chemistry, mathematics):
    total ← english + physics + chemistry + mathematics
    percentage ← total / 4.0
    grade ← calculate_grade(percentage)
    RETURN total, percentage, grade
END FUNCTION

```

Pseudocode 23.6 — calculate_grade() and the derived values

23.7 Generate Analytics

```

FUNCTION show_analytics(action):
    df ← READ the data file
    IF df is empty THEN
        SHOW INFO "No data available."
        RETURN
    END IF
    IF action = "Class Average" THEN result ← MEAN Percentage GROUPED BY Class
    IF action = "Topper" THEN result ← ROW with MAXIMUM Percentage
    IF action = "Lowest Scorer" THEN result ← ROW with MINIMUM Percentage
    IF action = "Top 5" THEN result ← FIVE ROWS with highest Percentage
    IF action = "Bottom 5" THEN result ← FIVE ROWS with lowest Percentage
    IF action = "Subject Avg" THEN result ← MEAN of the four subject columns
    IF action = "Pass Count" THEN result ← COUNT of rows whose Grade is not "F"
    IF action = "Fail Count" THEN result ← COUNT of rows whose Grade is "F"
    IF action = "Pass Pct" THEN result ← (Pass Count / TOTAL ROWS) * 100
    IF action = "Grade Dist" THEN result ← FREQUENCY of each Grade value
    SHOW INFORMATION titled action with result
END FUNCTION

```

Pseudocode 23.7 — show_analytics() — the ten action names are the exact button captions used in the interface

23.8 Generate Graph

```

FUNCTION plot_graph(graph_type):
    df ← READ the data file
    IF df is empty THEN
        SHOW INFO "No data to plot."
        RETURN
    END IF
    CREATE a figure of size 8 x 5 inches
    IF graph_type = "Top Students" THEN
        top ← FIVE ROWS with highest Percentage
        DRAW bar chart of top.Name against top.Percentage using five different colours
        SET title "Top 5 Students by Percentage", x-label "Student Name", y-label "Percentage (%)"
    ELSE IF graph_type = "Subject Average" THEN
        averages ← MEAN of English, Physics, Chemistry, Mathematics
        DRAW bar chart of averages using the colours skyblue, lightgreen, salmon, plum
        SET title "Class Average in Each Subject", x-label "Subjects", y-label "Average Marks"
    ELSE IF graph_type = "Grade Distribution" THEN
        counts ← FREQUENCY of each Grade value
        DRAW pie chart of counts with percentage labels, first slice at angle 140
        SET title "Grade Distribution of Students"
    END IF
    ADJUST the layout and DISPLAY the figure
END FUNCTION

```

Pseudocode 23.8 — plot_graph()

VIVA POINT — QUESTIONS ON THIS CHAPTER

Q. Why write pseudocode when the source code is already available?

Pseudocode separates the logic from the syntax. It makes it possible to check the sequence of steps — validate, then read, then modify, then save — without being distracted by Python spacing or function names, and it can be explained to a reader who does not know Python.

Q. Which pseudocode block shows the duplicate check?

The Add Student block tests the roll number before appending, and the Import CSV block compares the two sets of roll numbers. Both correspond to the two duplicate checks documented in Chapter 11.

KEY POINT — HOW TO EXPLAIN PSEUDOCODE IN A VIVA

Pseudocode states the logic without the syntax of any particular language. Reading it beside the source listing in Appendix A shows that each pseudocode step corresponds to one or two lines of Python — which is the quickest way to explain the program during the examination.

APPENDIX A

Complete Source Code

Project Program.py — 539 lines, reproduced in full without any modification

This appendix contains the complete source file `Project Program.py` — all 539 lines, in the original order, with the original indentation, comments and blank lines. Nothing has been shortened, rewritten or reformatted.

Region of the program	Source lines	Contents
A.1 Setup and Configuration	1 – 35	Imports, appearance mode, colour theme, data-file path, column list and creation of the window
A.2 Helper Functions	36 – 168	Grade calculation, form reset, table refresh, dashboard update, form reading with validation, and the calculation of total, percentage and grade
A.3 CRUD Operations	169 – 283	Add, update, delete, CSV import, search, and the selection handler that fills the form from the table
A.4 Analytics and Graph Functions	284 – 372	The ten analytics actions and the three Matplotlib charts
A.5 GUI Layout Design	373 – 539	Header, KPI cards, student form, buttons, search bar, table with scrollbars, analytics grid, chart buttons and the start of the event loop

Table A.1 — Map of the source file. Line numbers in the listing that follows are the original line numbers of the file. The listing is numbered 1 to 539; because the file does not end with a newline character, some line-counting tools report it as a 538-line file.

A.1 Setup and Configuration

The program begins by importing the six modules it uses, switching the interface to the dark appearance, defining the name and column structure of the data file and creating the application window. If the data file does not exist, an empty file with the thirteen column headings is written immediately.

```

1  import os
2  import tkinter as tk
3  from tkinter import ttk, messagebox
4  import customtkinter as ctk
5  import pandas as pd
6  import matplotlib.pyplot as plt
7
8  # =====
9  # SETUP AND CONFIGURATION
10 # =====
11
12 ctk.set_appearance_mode("Dark")
13 ctk.set_default_color_theme("blue")
14
15 FILE_NAME = os.path.join(os.path.dirname(__file__), "students.csv")
16 COLUMNS = [
17     "Roll Number",
18     "Name",
19     "Class",
20     "Section",
21     "Gender",
22     "Age",
23     "English",
24     "Physics",
25     "Chemistry",
26     "Mathematics",

```

```

27     "Total Marks",
28     "Percentage",
29     "Grade",
30 ]
31
32 if not os.path.exists(FILE_NAME):
33     df = pd.DataFrame(columns=COLUMNS)
34     df.to_csv(FILE_NAME, index=False)
35

```

A.1 Setup and Configuration — lines 1 to 35 of [Project Program.py](#). The code is reproduced exactly as written, including comments and blank lines.

A.2 Helper Functions

These functions are used by several operations: the grade threshold function, the form reset, the table refresh with its dashboard update, the form reader with its complete validation, and the calculation of the derived values.

```

36 app = ctk.CTk()
37 app.title("Student Management System")
38 app.geometry("1400x800")
39
40 # =====
41 # HELPER FUNCTIONS
42 # =====
43
44
45 def calculate_grade(percentage):
46     if percentage >= 90:
47         return "A+"
48     elif percentage >= 80:
49         return "A"
50     elif percentage >= 70:
51         return "B"
52     elif percentage >= 60:
53         return "C"
54     elif percentage >= 40:
55         return "D"
56     else:
57         return "F"
58
59
60 def clear_form():
61     roll_var.set("")
62     name_var.set("")
63     class_var.set("12")
64     section_var.set("A")
65     gender_var.set("Male")
66     age_var.set("")
67     eng_var.set("")
68     phy_var.set("")
69     chem_var.set("")
70     math_var.set("")
71
72
73 def refresh_table(search_df=None):
74     for row in tree.get_children():
75         tree.delete(row)
76
77     try:
78         if search_df is not None:
79             df = search_df
80         else:
81             df = pd.read_csv(FILE_NAME)
82
83         for _, row in df.iterrows():
84             tree.insert("", "end", values=list(row))
85

```

```

86         update_dashboard(df)
87     except Exception as e:
88         messagebox.showerror("Error", f"Could not load data: {e}")
89
90
91 def update_dashboard(df):
92     if df.empty:
93         kpi_total.configure(text="0")
94         kpi_avg.configure(text="0.0%")
95         kpi_topper.configure(text="N/A")
96         kpi_pass.configure(text="0.0%")
97         return
98
99     total_students = len(df)
100    avg_pct = df["Percentage"].mean()
101    topper_name = df.loc[df["Percentage"].idxmax()][ "Name" ]
102    pass_count = len(df[df["Grade"] != "F"])
103    pass_pct = (pass_count / total_students) * 100
104
105    kpi_total.configure(text=str(total_students))
106    kpi_avg.configure(text=f"{avg_pct:.2f}%")
107    kpi_topper.configure(text=str(topper_name))
108    kpi_pass.configure(text=f"{pass_pct:.2f}%")
109
110

```

A.2 Helper Functions (part 1) — lines 36 to 110 of [Project Program.py](#). The code is reproduced exactly as written, including comments and blank lines.

```

111 def get_form_data():
112     try:
113         roll = roll_var.get().strip()
114         name = name_var.get().strip()
115         cls = class_var.get()
116         sec = section_var.get()
117         gen = gender_var.get()
118         age = age_var.get().strip()
119         eng = eng_var.get().strip()
120         phy = phy_var.get().strip()
121         chem = chem_var.get().strip()
122         math = math_var.get().strip()
123
124         if not all([roll, name, cls, sec, gen, age, eng, phy, chem, math]):
125             messagebox.showwarning("Warning", "All fields are required!")
126             return None
127
128         roll = int(roll)
129         age = int(age)
130         eng = float(eng)
131         phy = float(phy)
132         chem = float(chem)
133         math = float(math)
134
135         if age <= 0 or age > 100:
136             messagebox.showerror("Error", "Please enter a valid age.")
137             return None
138
139         for mark in [eng, phy, chem, math]:
140             if mark < 0 or mark > 100:
141                 messagebox.showerror("Error", "Marks must be between 0 and 100.")
142                 return None
143
144         total = eng + phy + chem + math
145         percentage = total / 4.0
146         grade = calculate_grade(percentage)
147
148         return {
149             "Roll Number": roll,
150             "Name": name,
151             "Class": cls,
152             "Section": sec,
153             "Gender": gen,

```

```

154         "Age": age,
155         "English": eng,
156         "Physics": phy,
157         "Chemistry": chem,
158         "Mathematics": math,
159         "Total Marks": total,
160         "Percentage": percentage,
161         "Grade": grade,
162     }
163 except ValueError:
164     messagebox.showerror("Error", "Roll Number, Age and Marks must be valid numbers!")
165     return None
166
167
168 # =====

```

A.2 Helper Functions (part 2) — form reading and validation — lines 111 to 168 of `Project Program.py`. The code is reproduced exactly as written, including comments and blank lines.

A.3 CRUD Operations

The four data operations, the CSV import with its four checks, the search and the table-selection handler.

```

169 # CRUD OPERATIONS
170 # =====
171
172
173 def add_student():
174     data = get_form_data()
175     if data:
176         df = pd.read_csv(FILE_NAME)
177
178         if data["Roll Number"] in df["Roll Number"].values:
179             messagebox.showerror("Error", "Student with this Roll Number already exists!")
180             return
181
182         df = pd.concat([df, pd.DataFrame([data])], ignore_index=True)
183         df.to_csv(FILE_NAME, index=False)
184
185         messagebox.showinfo("Success", "Student Added Successfully!")
186         clear_form()
187         refresh_table()
188
189
190 def update_student():
191     data = get_form_data()
192     if data:
193         df = pd.read_csv(FILE_NAME)
194         roll = data["Roll Number"]
195
196         if roll not in df["Roll Number"].values:
197             messagebox.showerror("Error", "Roll Number not found in database!")
198             return
199
200         df.loc[df["Roll Number"] == roll, list(data.keys())] = list(data.values())
201         df.to_csv(FILE_NAME, index=False)
202
203         messagebox.showinfo("Success", "Student Updated Successfully!")
204         refresh_table()
205
206
207 def delete_student():
208     try:
209         roll = int(roll_var.get().strip())
210         df = pd.read_csv(FILE_NAME)
211
212         if roll not in df["Roll Number"].values:
213             messagebox.showerror("Error", "Roll Number not found!")
214             return
215

```

```

216         confirm = messagebox.askyesno("Confirm Delete", "Are you sure you want to delete this student?")
217         if confirm:
218             df = df[df["Roll Number"] != roll]
219             df.to_csv(FILE_NAME, index=False)
220             messagebox.showinfo("Success", "Student Deleted Successfully!")
221             clear_form()
222             refresh_table()
223     except ValueError:
224         messagebox.showerror("Error", "Please enter a valid Roll Number to delete.")
225
226
227 def import_csv():
228     file_path = tk.filedialog.askopenfilename(
229         title="Select CSV File",
230         filetypes=[("CSV Files", "*.csv"), ("All Files", "*.*")]
231     )
232     if not file_path:
233         return
234
235     try:
236         import_df = pd.read_csv(file_path)
237
238         missing_cols = set(COLUMNS) - set(import_df.columns)
239         if missing_cols:
240             messagebox.showerror("Error", f"Missing required columns: {'', '.join(missing_cols)}")
241             return
242
243         import_df = import_df[COLUMNS]
244
245         for col in ["Roll Number", "Age"]:
246             import_df[col] = pd.to_numeric(import_df[col], errors="coerce")
247         for col in ["English", "Physics", "Chemistry", "Mathematics", "Total Marks", "Percentage"]:
248             import_df[col] = pd.to_numeric(import_df[col], errors="coerce")
249
250         if import_df.isnull().any().any():
251             messagebox.showerror("Error", "CSV contains invalid or missing data in required fields.")
252             return
253
254         existing_df = pd.read_csv(FILE_NAME)
255         duplicate_rolls = set(import_df["Roll Number"]).intersection(set(existing_df["Roll Number"]))
256         if duplicate_rolls:
257             messagebox.showerror("Error", f"Duplicate Roll Numbers found: {'', '.join(map(str,
duplicate_rolls))}")
258             return
259
260         combined_df = pd.concat([existing_df, import_df], ignore_index=True)
261         combined_df.to_csv(FILE_NAME, index=False)
262
263         messagebox.showinfo("Success", f"Imported {len(import_df)} students successfully!")
264         refresh_table()
265     except Exception as e:
266         messagebox.showerror("Error", f"Failed to import CSV: {e}")
267
268
269 def search_student():
270     query = search_var.get().strip().lower()
271     if not query:
272         refresh_table()
273         return
274
275     df = pd.read_csv(FILE_NAME)
276     filtered_df = df[
277         df["Roll Number"].astype(str).str.contains(query) |
278         df["Name"].str.lower().str.contains(query)
279     ]
280     refresh_table(filtered_df)
281
282
283 def fill_form_from_tree(event):

```

A.3 CRUD Operations — lines 169 to 283 of [Project Program.py](#). The code is reproduced exactly as written, including comments and blank lines.

A.4 Analytics and Graph Functions

The single analytics function containing the ten actions, followed by the chart function containing the three Matplotlib figures.

```

284     selected_item = tree.selection()
285     if selected_item:
286         values = tree.item(selected_item[0], "values")
287         if values:
288             roll_var.set(values[0])
289             name_var.set(values[1])
290             class_var.set(values[2])
291             section_var.set(values[3])
292             gender_var.set(values[4])
293             age_var.set(values[5])
294             eng_var.set(values[6])
295             phy_var.set(values[7])
296             chem_var.set(values[8])
297             math_var.set(values[9])
298
299
300     # =====
301     # ANALYTICS & GRAPHS FUNCTIONS
302     # =====
303
304
305     def show_analytics(action):
306         df = pd.read_csv(FILE_NAME)
307         if df.empty:
308             messagebox.showinfo("Info", "No data available.")
309             return
310
311         result = ""
312         if action == "Class Average":
313             result = df.groupby("Class")["Percentage"].mean().to_string()
314         elif action == "Topper":
315             topper = df.loc[df["Percentage"].idxmax()]
316             result = f"Name: {topper['Name']}\nPercentage: {topper['Percentage']}%\nClass: {topper['Class']}"
317         elif action == "Lowest Scorer":
318             lowest = df.loc[df["Percentage"].idxmin()]
319             result = f"Name: {lowest['Name']}\nPercentage: {lowest['Percentage']}%\nClass: {lowest['Class']}"
320         elif action == "Top 5":
321             top5 = df.nlargest(5, "Percentage")[["Name", "Class", "Percentage"]]
322             result = top5.to_string(index=False)
323         elif action == "Bottom 5":
324             bot5 = df.nsmallest(5, "Percentage")[["Name", "Class", "Percentage"]]
325             result = bot5.to_string(index=False)
326         elif action == "Subject Avg":
327             subs = df[["English", "Physics", "Chemistry", "Mathematics"]].mean()
328             result = subs.to_string()
329         elif action == "Pass Count":
330             count = len(df[df["Grade"] != "F"])
331             result = f"Total Passed: {count}"
332         elif action == "Fail Count":
333             count = len(df[df["Grade"] == "F"])
334             result = f"Total Failed: {count}"
335         elif action == "Pass Pct":
336             count = len(df[df["Grade"] != "F"])
337             pct = (count / len(df)) * 100
338             result = f"Pass Percentage: {pct:.2f}%"
339         elif action == "Grade Dist":
340             dist = df["Grade"].value_counts()
341             result = dist.to_string()
342
343         messagebox.showinfo(action, result)
344
345
346     def plot_graph(graph_type):
347         df = pd.read_csv(FILE_NAME)

```

```

348     if df.empty:
349         messagebox.showinfo("Info", "No data to plot.")
350         return
351
352     plt.figure(figsize=(8, 5))
353
354     if graph_type == "Top Students":
355         top_students = df.nlargest(5, "Percentage")
356         plt.bar(top_students["Name"], top_students["Percentage"], color=["blue", "green", "red",
"purple", "orange"])
357         plt.title("Top 5 Students by Percentage")
358         plt.xlabel("Student Name")
359         plt.ylabel("Percentage (%)")
360     elif graph_type == "Subject Average":
361         averages = df[["English", "Physics", "Chemistry", "Mathematics"]].mean()
362         plt.bar(averages.index, averages.values, color=["skyblue", "lightgreen", "salmon", "plum"])
363         plt.title("Class Average in Each Subject")
364         plt.xlabel("Subjects")
365         plt.ylabel("Average Marks")
366     elif graph_type == "Grade Distribution":
367         grade_counts = df["Grade"].value_counts()
368         plt.pie(grade_counts.values, labels=grade_counts.index, autopct="%1.1f%%", startangle=140)
369         plt.title("Grade Distribution of Students")
370
371     plt.tight_layout()
372     plt.show()

```

A.4 Analytics and Graph Functions — lines 284 to 372 of [Project Program.py](#). The code is reproduced exactly as written, including comments and blank lines.

A.5 GUI Layout Design

The construction of the interface, written from top to bottom: header, dashboard cards, form, buttons, search area, record table with scrollbars, analytics grid and chart buttons, ending with the first table refresh and the start of the Tkinter event loop.

```

373
374
375 # =====
376 # GUI LAYOUT DESIGN
377 # =====
378
379 header_frame = ctk.CTkFrame(app, fg_color="transparent")
380 header_frame.pack(fill="x", pady=10)
381
382 ctk.CTkLabel(header_frame, text="STUDENT MANAGEMENT SYSTEM", font=("Arial", 32, "bold")).pack()
383 ctk.CTkLabel(header_frame, text="CBSE Informatics Practices Project", font=("Arial", 16)).pack()
384
385 kpi_frame = ctk.CTkFrame(app)
386 kpi_frame.pack(fill="x", padx=20, pady=10)
387
388
389 def create_kpi_card(parent, title):
390     card = ctk.CTkFrame(parent, fg_color="#2b2b2b", corner_radius=10)
391     card.pack(side="left", fill="both", expand=True, padx=10, pady=10)
392     ctk.CTkLabel(card, text=title, font=("Arial", 14)).pack(pady=(10, 0))
393     val_label = ctk.CTkLabel(card, text="-", font=("Arial", 24, "bold"), text_color="#1f6aa5")
394     val_label.pack(pady=(5, 10))
395     return val_label
396
397
398 kpi_total = create_kpi_card(kpi_frame, "Total Students")
399 kpi_avg = create_kpi_card(kpi_frame, "Average Percentage")
400 kpi_topper = create_kpi_card(kpi_frame, "Topper Name")
401 kpi_pass = create_kpi_card(kpi_frame, "Pass Percentage")
402
403 main_frame = ctk.CTkFrame(app, fg_color="transparent")
404 main_frame.pack(fill="both", expand=True, padx=20, pady=10)
405

```

```

406 form_frame = ctk.CTkFrame(main_frame, width=350)
407 form_frame.pack(side="left", fill="y", padx=(0, 10))
408
409 ctk.CTkLabel(form_frame, text="Student Details Form", font=("Arial", 20, "bold")).pack(pady=15)
410
411 roll_var = tk.StringVar()
412 name_var = tk.StringVar()
413 class_var = tk.StringVar(value="12")
414 section_var = tk.StringVar(value="A")
415 gender_var = tk.StringVar(value="Male")
416 age_var = tk.StringVar()
417 eng_var = tk.StringVar()
418 phy_var = tk.StringVar()
419 chem_var = tk.StringVar()
420 math_var = tk.StringVar()
421
422
423 def make_entry(parent, label, variable, is_combo=False, combo_values=None):
424     frame = ctk.CTkFrame(parent, fg_color="transparent")
425     frame.pack(fill="x", padx=20, pady=5)
426     ctk.CTkLabel(frame, text=label, width=100, anchor="w").pack(side="left")
427     if is_combo:
428         comp = ctk.CTkComboBox(frame, variable=variable, values=combo_values, width=180)
429     else:
430         comp = ctk.CTkEntry(frame, textvariable=variable, width=180)
431     comp.pack(side="right")
432
433
434 make_entry(form_frame, "Roll Number:", roll_var)
435 make_entry(form_frame, "Student Name:", name_var)
436 make_entry(form_frame, "Class:", class_var, True, ["11", "12"])
437 make_entry(form_frame, "Section:", section_var, True, ["A", "B", "C", "D"])
438 make_entry(form_frame, "Gender:", gender_var, True, ["Male", "Female", "Other"])
439 make_entry(form_frame, "Age:", age_var)
440 make_entry(form_frame, "English Marks:", eng_var)
441 make_entry(form_frame, "Physics Marks:", phy_var)
442 make_entry(form_frame, "Chemistry Marks:", chem_var)
443 make_entry(form_frame, "Maths Marks:", math_var)
444
445 btn_frame = ctk.CTkFrame(form_frame, fg_color="transparent")
446 btn_frame.pack(pady=20)
447
448 ctk.CTkButton(btn_frame, text="Add", width=80, command=add_student).grid(row=0, column=0, padx=5,
449     pady=5)
450 ctk.CTkButton(btn_frame, text="Update", width=80, command=update_student).grid(row=0, column=1, padx=5,
451     pady=5)
452 ctk.CTkButton(btn_frame, text="Delete", width=80, fg_color="red", hover_color="darkred",
453     command=delete_student).grid(row=0, column=2, padx=5, pady=5)
454 ctk.CTkButton(btn_frame, text="Import CSV", width=80, fg_color="#2e7d32", hover_color="#1b5e20",
455     command=import_csv).grid(row=0, column=3, padx=5, pady=5)
456 ctk.CTkButton(btn_frame, text="Clear Fields", width=260, fg_color="gray",
457     command=clear_form).grid(row=1, column=0, colspan=4, pady=5)
458
459
460 right_frame = ctk.CTkFrame(main_frame, fg_color="transparent")
461 right_frame.pack(side="right", fill="both", expand=True)
462
463 search_frame = ctk.CTkFrame(right_frame, fg_color="transparent")
464 search_frame.pack(fill="x", pady=(0, 10))
465
466 search_var = tk.StringVar()
467 ctk.CTkEntry(search_frame, textvariable=search_var, placeholder_text="Search by Name or Roll Number",
468     width=300).pack(side="left", padx=(0, 10))
469 ctk.CTkButton(search_frame, text="Search", width=100, command=search_student).pack(side="left", padx=5)
470 ctk.CTkButton(search_frame, text="View All", width=100, fg_color="gray", command=lambda:
471     [search_var.set(""), refresh_table()]).pack(side="left", padx=5)
472
473
474 style = ttk.Style()
475 style.theme_use("default")
476 style.configure("Treeview", background="#2b2b2b", foreground="white", rowheight=25,
477     fieldbackground="#2b2b2b", borderwidth=0)
478 style.map("Treeview", background=[("selected", "#1f6aa5")])
479 style.configure("Treeview.Heading", background="#1f1f1f", foreground="white", font=("Arial", 10,
480     "bold"))

```

```

470
471 tree_frame = ctk.CTkFrame(right_frame)
472 tree_frame.pack(fill="both", expand=True)
473
474 scroll_y = ttk.Scrollbar(tree_frame, orient="vertical")
475 scroll_x = ttk.Scrollbar(tree_frame, orient="horizontal")
476
477 tree = ttk.Treeview(tree_frame, columns=COLUMNS, show="headings", yscrollcommand=scroll_y.set,
478 xscrollcommand=scroll_x.set)
479 scroll_y.config(command=tree.yview)
480 scroll_x.config(command=tree.xview)
481
482 scroll_y.pack(side="right", fill="y")
483 scroll_x.pack(side="bottom", fill="x")
484 tree.pack(fill="both", expand=True)
485
486 for col in COLUMNS:
487     tree.heading(col, text=col)
488     tree.column(col, width=90, anchor="center")
489
490 tree.bind("<<TreeviewSelect>>", fill_form_from_tree)
491
492 bottom_panel = ctk.CTkFrame(right_frame, fg_color="transparent")
493 bottom_panel.pack(fill="x", pady=10)
494
495 analytics_lbl = ctk.CTkLabel(bottom_panel, text="Data Analytics & Reports", font=("Arial", 16, "bold"))
496 analytics_lbl.pack(pady=(0, 5))
497
498 an_frame = ctk.CTkFrame(bottom_panel, fg_color="transparent")
499 an_frame.pack()
500
501 analytics_list = [
502     "Class Average",
503     "Topper",
504     "Lowest Scorer",
505     "Top 5",
506     "Bottom 5",
507     "Subject Avg",
508     "Pass Count",
509     "Fail Count",
510     "Pass Pct",
511     "Grade Dist",
512 ]
513
514 row_val = 0
515 col_val = 0
516 for item in analytics_list:
517     ctk.CTkButton(
518         an_frame,
519         text=item,
520         width=110,
521         fg_color="#3a3a3a",
522         hover_color="#4a4a4a",
523         command=lambda txt=item: show_analytics(txt),
524     ).grid(row=row_val, column=col_val, padx=5, pady=5)
525     col_val += 1
526     if col_val > 4:
527         col_val = 0
528         row_val += 1
529
530 graph_frame = ctk.CTkFrame(bottom_panel, fg_color="transparent")
531 graph_frame.pack(pady=10)
532
533 ctk.CTkButton(graph_frame, text="📊 Top Students Chart", width=180, fg_color="#1f6aa5", command=lambda:
534 plot_graph("Top Students")).pack(side="left", padx=10)
535 ctk.CTkButton(graph_frame, text="📊 Subject Average Chart", width=180, fg_color="#1f6aa5",
536 command=lambda: plot_graph("Subject Average")).pack(side="left", padx=10)
537 ctk.CTkButton(graph_frame, text="🥧 Grade Distribution Pie", width=180, fg_color="#1f6aa5",
538 command=lambda: plot_graph("Grade Distribution")).pack(side="left", padx=10)
539
540 refresh_table()
541
542 if __name__ == "__main__":

```

```
539 | app.mainloop()
```

A.5 GUI Layout Design — lines 373 to 539 of `Project Program.py`. The code is reproduced exactly as written, including comments and blank lines.

A.6 Running the submitted program

The program is a single file. Once the three required packages are installed, the complete application can be started with one command from the folder that contains the source file.

```
1 | python -m pip install customtkinter pandas matplotlib
2 | python "Project Program.py"
```

Listing A.1 — installation of the three external packages and start-up of the application. Tkinter and the os module are part of the standard Python installation.

On the first run the program creates an empty `students.csv` file beside the source file. The twenty-record sample dataset supplied with this project can then be loaded with the Import CSV button, which makes all ten analytics operations and the three charts available for demonstration.

Step	Action	What should appear
1	Run the program	The window opens with four KPI cards showing 0, 0.0%, N/A and 0.0%, and an empty table
2	Click Import CSV and select <code>Student.csv</code>	"Imported 20 students successfully!" and twenty rows in the table
3	Observe the dashboard	20 · 77.44% · Sneha Reddy · 95.00%
4	Type reddy in the search box and click Search	One matching record; the KPI cards change to the filtered values
5	Click View All	The complete table returns
6	Click any analytics button	A result dialog for that metric
7	Click Top Students Chart	A Matplotlib window with the bar chart of the five best percentages

Table A.2 — a short sequence that demonstrates every major module of the application in a few minutes.

KEY POINT — COMPLETENESS OF THIS APPENDIX

The listing above contains the complete source file — 539 lines in 5 regions, reproduced from `Project Program.py` without any change, including all comments and blank lines. The six modules imported on lines 1 to 6 are the only external modules used by the program.

APPENDIX B

Sample Dataset

Student.csv — twenty dummy student records supplied with the project

This appendix reproduces the supplied dataset `Student.csv` exactly as it was received: twenty dummy student records in the thirteen-column format of the application. The file is ready to load through the Import CSV button without any editing.

SAMPLE / DUMMY DATA — FOR DEMONSTRATION

The names, marks and other values in this file are artificial. They do not represent real students, and the marks do not correspond to any actual examination.

B.1 Raw file contents

The complete file is printed below, line by line, in its original order. Line 1 holds the column headings and the following twenty lines hold one student record each.

```
1 Roll Number,Name,Class,Section,Gender,Age,English,Physics,Chemistry,Mathematics,Total
  Marks,Percentage,Grade
2 101,Aarav Sharma,12,A,Male,17,85,92,88,90,355,88.75,A
3 102,Priya Patel,12,A,Female,17,78,85,82,79,324,81.00,A
4 103,Rohan Gupta,12,B,Male,18,92,95,90,93,370,92.50,A+
5 104,Anjali Singh,12,B,Female,17,70,65,72,68,275,68.75,C
6 105,Vikram Mehta,11,A,Male,16,88,91,89,87,355,88.75,A
7 106,Sneha Reddy,11,A,Female,16,95,93,96,94,378,94.50,A+
8 107,Arjun Kumar,11,B,Male,17,45,50,48,52,195,48.75,D
9 108,Meera Joshi,11,B,Female,16,82,79,81,80,322,80.50,A
10 109,Karan Verma,12,C,Male,18,60,58,62,55,235,58.75,D
11 110,Neha Agarwal,12,C,Female,17,90,88,92,89,359,89.75,A
12 111,Aditya Nair,11,C,Male,16,75,72,78,74,299,74.75,B
13 112,Kavya Iyer,11,C,Female,16,88,85,90,87,350,87.50,A
14 113,Rahul Shah,12,D,Male,17,55,60,58,62,235,58.75,D
15 114,Pooja Menon,12,D,Female,17,83,80,85,82,330,82.50,A
16 115,Siddharth Rao,11,D,Male,16,91,89,93,90,363,90.75,A+
17 116,Tanvi Desai,11,D,Female,16,68,70,65,67,270,67.50,C
18 117,Nikhil Bansal,12,A,Male,18,40,38,42,35,155,38.75,F
19 118,Divya Kapoor,12,A,Female,17,79,76,80,77,312,78.00,B
20 119,Manish Tiwari,11,B,Male,16,86,84,88,85,343,85.75,A
21 120,Riya Malhotra,11,B,Female,16,93,91,94,92,370,92.50,A+
```

Figure B.1 — complete contents of `Student.csv` — 1 heading line and 20 data lines (21 lines in total), unchanged.

B.2 Consistency of the derived columns

The table below verifies that the three calculated columns of the supplied dataset follow exactly the rules implemented in the program: the total marks equal the sum of the four subject marks, the percentage equals the total divided by four, and the grade matches the thresholds of `calculate_grade()`.

Roll	Name	Sum of four subjects	Total Marks in file	Match	Total / 4	Percentage in file	Match	Grade recomputed	Grade in file	Match
101	Aarav Sharma	355	355	Yes	88.75	88.75	Yes	A	A	Yes
102	Priya Patel	324	324	Yes	81.00	81.00	Yes	A	A	Yes

Roll	Name	Sum of four subjects	Total Marks in file	Match	Total / 4	Percentage in file	Match	Grade recomputed	Grade in file	Match
103	Rohan Gupta	370	370	Yes	92.50	92.50	Yes	A+	A+	Yes
104	Anjali Singh	275	275	Yes	68.75	68.75	Yes	C	C	Yes
105	Vikram Mehta	355	355	Yes	88.75	88.75	Yes	A	A	Yes
106	Sneha Reddy	378	378	Yes	94.50	94.50	Yes	A+	A+	Yes
107	Arjun Kumar	195	195	Yes	48.75	48.75	Yes	D	D	Yes
108	Meera Joshi	322	322	Yes	80.50	80.50	Yes	A	A	Yes
109	Karan Verma	235	235	Yes	58.75	58.75	Yes	D	D	Yes
110	Neha Agarwal	359	359	Yes	89.75	89.75	Yes	A	A	Yes
111	Aditya Nair	299	299	Yes	74.75	74.75	Yes	B	B	Yes
112	Kavya Iyer	350	350	Yes	87.50	87.50	Yes	A	A	Yes
113	Rahul Shah	235	235	Yes	58.75	58.75	Yes	D	D	Yes
114	Pooja Menon	330	330	Yes	82.50	82.50	Yes	A	A	Yes
115	Siddharth Rao	363	363	Yes	90.75	90.75	Yes	A+	A+	Yes
116	Tanvi Desai	270	270	Yes	67.50	67.50	Yes	C	C	Yes
117	Nikhil Bansal	155	155	Yes	38.75	38.75	Yes	F	F	Yes
118	Divya Kapoor	312	312	Yes	78.00	78.00	Yes	B	B	Yes
119	Manish Tiwari	343	343	Yes	85.75	85.75	Yes	A	A	Yes
120	Riya Malhotra	370	370	Yes	92.50	92.50	Yes	A+	A+	Yes

Table B.1 — verification of the total, percentage and grade of every record in the supplied dataset. All twenty records satisfy the three rules, which is why the dataset can be imported directly.

B.3 How to load this dataset

1. Start the application. An empty `students.csv` file is created automatically in the program folder if it is not already present.

2. Click **Import CSV** and select `Student.csv`.
3. The program checks the columns, the values and the roll numbers, then reports "Imported 20 students successfully!".
4. The table, the four dashboard cards, all ten analytics operations and the three charts now work on these twenty records.

KEY POINT — WHY THIS DATASET QUALIFIES AS PLUG-AND-PLAY

Its column headings match the thirteen names in the `COLUMNS` constant exactly, every numeric field contains a valid number, no value is missing and no roll number is repeated — the three conditions that the import function tests before accepting a file.

B.4 Composition of the sample dataset

The tables below summarise the twenty records of the supplied file. They were computed from the same CSV data that the application reads, and they are useful for checking that an import has completed correctly.

Class	Records	Average %	Highest %	Lowest %	Sections
11	10	81.12	94.50	48.75	A, B, C, D
12	10	73.75	92.50	38.75	A, B, C, D
All	20	77.44	94.50	38.75	A, B, C, D

Group	Boys	Girls	Total	Records with grade A+ or A
Class 11	5	5	10	7
Class 12	5	5	10	5
Total	10	10	20	12

Table B.2 — Composition of the sample dataset. The file contains ten records of Class 11 and ten of Class 12, evenly divided between boys and girls and distributed over sections A, B, C and D, which makes it suitable for demonstrating the class-wise analytics. Seven of the twelve records with grade A+ or A belong to Class 11, which explains the higher class average of Class 11.

Figure B.3 — The twenty records are the data behind the dashboard values, the analytics results and the three charts presented in this report.

CHAPTER 26

Project Limitations

What the current implementation does not attempt to do

The limitations below follow directly from the design of the submitted application. They describe the boundaries of the implementation, not defects: each one is a consequence of a deliberate simplification that keeps the project suitable for its academic purpose.

Area	Limitation	Consequence
Storage format	Data is stored in a single CSV file rather than in a relational database.	There is no enforcement of data types or unique keys by the storage layer itself; all such checks are performed by the program code.
Application type	The application is a desktop program that runs on one computer at a time.	Records cannot be accessed from another machine or from a mobile device.
Authentication	There is no login or user-role module.	Any person who can open the program can view and modify all records.
Cloud synchronisation	No cloud storage or synchronisation is implemented.	The data file must be copied manually if a second computer has to be used.
Multi-user access	There is no network or collaboration support.	Two users cannot work on the same data file at the same time.
Analytics basis	All summaries are computed from the records currently present in the loaded CSV file.	A wrong or incomplete record directly affects the averages, the pass percentage and the charts.
Graphs	Charts are drawn only when valid data is available.	The chart buttons report "No data to plot." when the file is empty, because there is nothing to represent.
Grade model	The grade depends on the percentage of four subjects; there is no subject-wise pass requirement.	A student with a low mark in one subject but a good overall percentage is still shown as passed, because the pass rule is based only on the overall grade.

Table 26.1 — Limitations of the current implementation and their practical consequences.

KEY POINT — HOW TO ANSWER 'WHAT ARE THE LIMITATIONS?'

The honest answer is that the project stores data in a CSV file for one user on one computer, and that every analysis depends on the accuracy of the stored records. Mentioning the missing login module and the absence of cloud or multi-user support at the same time gives a complete and accurate picture.

CHAPTER 27

Future Enhancements

Proposed features for a later version — not part of the current implementation

These are proposed future enhancements and are not part of the current implementation. Everything listed on this page is an idea for a later version of the project. None of these features exists in the submitted source code, and none of them is required for the present academic submission.

SQL database integration

Move the records from the CSV file to a database such as SQLite or MySQL, so that queries, indexes and constraints are handled by the database engine.

Login and role-based access

Add a user module so that only authorised staff can add, modify or delete records, with separate rights for viewing and editing.

Attendance module

Record daily attendance and include it in the analysis, for example monthly attendance percentages per student and per class.

Subject-wise detailed reports

Produce a separate report for each subject with the highest, lowest and average marks, and a list of students below a chosen mark.

Export to PDF

Generate a printable report card or class result sheet directly from the application.

Automated report cards

Combine marks, grades and remarks into a formatted report card for each student at the end of a term.

Backup and restore

Create a dated copy of the data file automatically and provide a restore option.

Student profile photographs

Store a photograph with each record and display it beside the form when a row is selected.

Password protection

Protect the data file or the application itself with a password.

Advanced filtering

Filter records by class, section, gender or a range of marks, and sort the table on any column.

More statistical visualisations

Add histograms, scatter plots of subject-versus-subject performance and comparison charts between classes.

Executable application

Package the program with a tool such as PyInstaller so that it can be run on a computer that does not have Python installed.

Table 27.1 — Ideas for possible future versions of the Student Management System.

VIVA POINT — QUESTIONS ON THIS CHAPTER

Q. Which enhancement would need the smallest change to the current code?

Advanced filtering and sorting, because the table is already filled by a single function that accepts a DataFrame. A filter would only change which DataFrame is passed to that function.

Q. Which enhancement would need the largest change?

Login and role-based access, because the program currently has no user concept at all — the interface, the storage functions and the data file would all have to take the identity of the user into account.

KEY POINT — HOW TO PRESENT FUTURE SCOPE

Future scope should be described as a plan, not as a feature. A useful answer in the viva is to connect one item to the present design — for example, “the CSV storage could be replaced by an SQLite database, because all data access in the program already goes through Pandas, so only the storage functions in the source would need to change.”

CHAPTER 28

Conclusion

What the project achieves and what was demonstrated

The Student Management System fulfils the objectives set at the beginning of the project. It stores student records in a structured CSV file, manages them through a complete set of create, read, update and delete operations, validates every input before it is saved, calculates the total marks, percentage and grade automatically, produces ten analytical summaries, and presents three charts drawn from the same data.

The project demonstrates the practical use of Python for application development rather than for isolated programs. The interface, the validation, the storage and the analysis are written as separate functions, and the program is controlled through an event loop that reacts to what the user does. Writing the application in this way showed how a larger program is built from small, testable pieces.

The data-handling part of the project relies on Pandas. Reading a CSV file into a DataFrame, appending a record with concatenation, replacing the values of a row through a boolean condition, removing a record by filtering, and computing averages, extremes and frequencies — each of these operations required one expression, and the same DataFrame served both the on-screen table and the analysis.

Validation proved to be the part of the project that demanded the most care. Rejecting a blank field, a non-numeric mark, an out-of-range value, a duplicate roll number or an incomplete imported file is what keeps the CSV file consistent, and the confirmation dialog before deletion protects the only copy of the data from accidental loss.

The analytics and the charts complete the application by turning stored records into information — the class average, the topper, the pass percentage and the grade distribution of the demonstration dataset, together with the subject-wise averages and the top and bottom performers. Taken together, the project is a small but complete record-management and reporting application developed with the tools studied in Informatics Practices.

Summary of achievement — a working Python desktop application with a graphical interface, a CSV data file, complete CRUD functionality with validation, ten analytical operations and three Matplotlib charts, documented in this report from the actual source code and demonstrated with a twenty-record sample dataset.

KEY POINT — STATEMENT OF SCOPE

The project is submitted as an academic application for the CBSE Informatics Practices course. It has been developed and tested on a single computer, and it is documented here exactly as implemented, including the calculated values produced from the supplied sample dataset.

CHAPTER 29

Learning Outcomes

What the project taught, in the student's own words

Working on this project was as much a learning exercise as it was a coding task. The list below records what the project taught, in the order in which the difficulty was encountered.

Python GUI programming

Building a complete interface from widgets — frames, labels, entries, combo boxes and buttons — and understanding how layout managers place them inside a window.

Event-driven programming

Understanding that a graphical program waits for events and reacts to them, and that each button is linked to the function that performs its work.

File handling

Creating a file when it does not exist, reading it, writing it back after every change, and keeping its structure fixed through a column definition.

Tabular data processing

Working with rows and columns logically instead of treating data as plain text, and selecting the columns that an operation actually needs.

Pandas operations

Reading and writing CSV files, concatenating DataFrames, updating values through a boolean condition, filtering rows, and computing mean, maximum, minimum and frequency with single expressions.

Data validation

Deciding what a valid record is, checking every field before saving it, and reporting the problem to the user in language that explains what to correct.

CRUD implementation

Translating the four standard data operations into working code and understanding why each one must leave the stored file consistent.

Analytical calculations

Converting raw marks into meaningful indicators such as averages, pass percentage and grade distribution, and interpreting the results.

Chart generation

Choosing a suitable chart type for a question, preparing the data for it and labelling the result so that it communicates clearly.

Debugging and problem solving

Locating errors caused by invalid input, empty data files and mismatched columns, and correcting them by adding the checks that were missing.

Software design

Dividing a program into functions with a single responsibility each, and keeping the interface, the logic and the storage separate so that a change in one part does not break the others.

Table 29.1 — Skills gained during the design, development, testing and documentation of the project.

KEY POINT — THE HARDEST PART

Keeping the data file consistent was more difficult than writing the interface. An interface change is visible immediately, but a wrong record silently affects every average and every chart. This is why the validation checks and the duplicate roll-number checks were added before the analytics were written.

KEY POINT — THE MOST USEFUL HABIT

Testing one operation at a time with a small dataset — and deliberately entering wrong values — revealed more problems than reading the code again. The twenty-record sample dataset made this kind of quick testing possible.

CHAPTER 30

Bibliography / References

Documentation consulted for the technologies used

The references below are the standard documentation sources that were consulted while developing the project. They are the primary references for the libraries actually used in the source code.

Author / organisation	Reference	Relevance to this project
Python Software Foundation	The Python Tutorial and the Python 3 Language Reference	Official documentation of the Python programming language used to write the complete application.
Python Software Foundation	tkinter — Python interface to Tcl/Tk	Reference for the table widget, the scrollbars, the file dialog and the message boxes used in the project.
CustomTkinter project	CustomTkinter Documentation	Reference for the modern widgets, the appearance modes and the colour themes of the interface.
Pandas development team	pandas — Python Data Analysis Library documentation	Reference for reading and writing CSV files and for the DataFrame operations used in the CRUD and analytics functions.
Matplotlib development team	Matplotlib: Visualization with Python — documentation	Reference for the bar charts and the pie chart generated by the project.
CBSE	Informatics Practices (Code 065) — Class XII curriculum and practical guidelines	Defines the scope of the project work and the expectations of the practical examination.

Table 30.1 — Documentation sources consulted during the development of the project.

VIVA POINT — QUESTIONS ON THIS CHAPTER

Q. Which documentation was needed most often?

The Pandas documentation, because every data operation in the program — reading, combining, filtering, updating and summarising — uses a Pandas method whose exact behaviour had to be checked.

Q. Is any third-party library used outside this list?

No. The program imports only `os`, `tkinter`, `customtkinter`, `pandas` and `matplotlib.pyplot`.

KEY POINT — HOW TO USE THESE REFERENCES

Each entry corresponds to one part of the project — Python for the program itself, Tkinter and CustomTkinter for the interface, Pandas for the data processing and Matplotlib for the charts. No book or website other than the official documentation of these tools was required.

CHAPTER 31

Final Project Summary

The project on a single page

A single-page summary of the submitted project.

PROJECT Student Management System Student Records, Academic Analytics & Data Visualization Class XII · Informatics Practices · 2026-27 Adani Public School, Mundra	TECHNOLOGY Python + CustomTkinter + Tkinter + Pandas + Matplotlib + CSV Data storage: a single CSV file with thirteen columns Interface: dark themed desktop window of 1400 × 800 pixels
CORE MODULES CRUD Search Import Analytics Visualisation Fifteen functions in a single source file of 539 lines, documented completely in Appendix A.	DATA 20 sample demonstration records in Student.csv Imported through the application itself, producing an average of 77.44%, a pass percentage of 95.00% and the grade distribution shown in Chapter 18.

Table 31.1 — Project identity, technology and data at a glance.

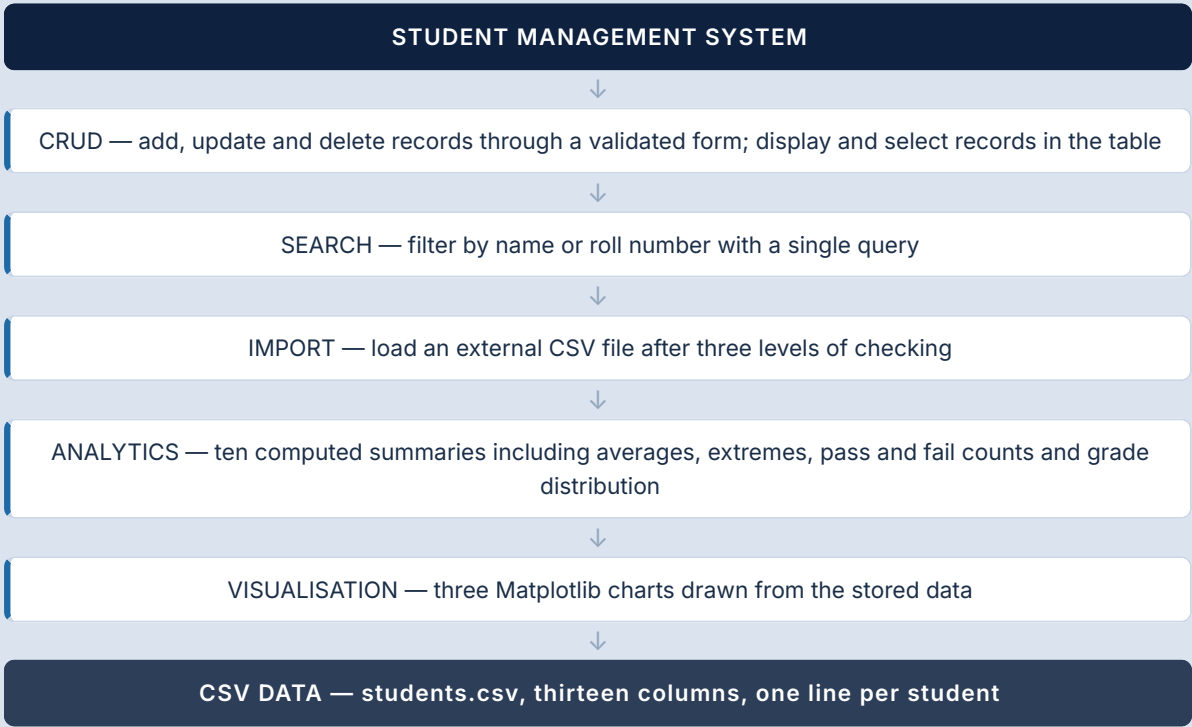


Figure 31.2 — The major modules of the project around the central data file.

539 LINES OF SOURCE CODE	13 DATA COLUMNS	10 ANALYTICS OPERATIONS	3 CHARTS	20 SAMPLE RECORDS
------------------------------------	---------------------------	-----------------------------------	--------------------	-----------------------------

Table 31.1 — The project in numbers. The complete source listing appears in Appendix A and the sample dataset in Appendix B.